I/O, Storage Devices, and File Systems

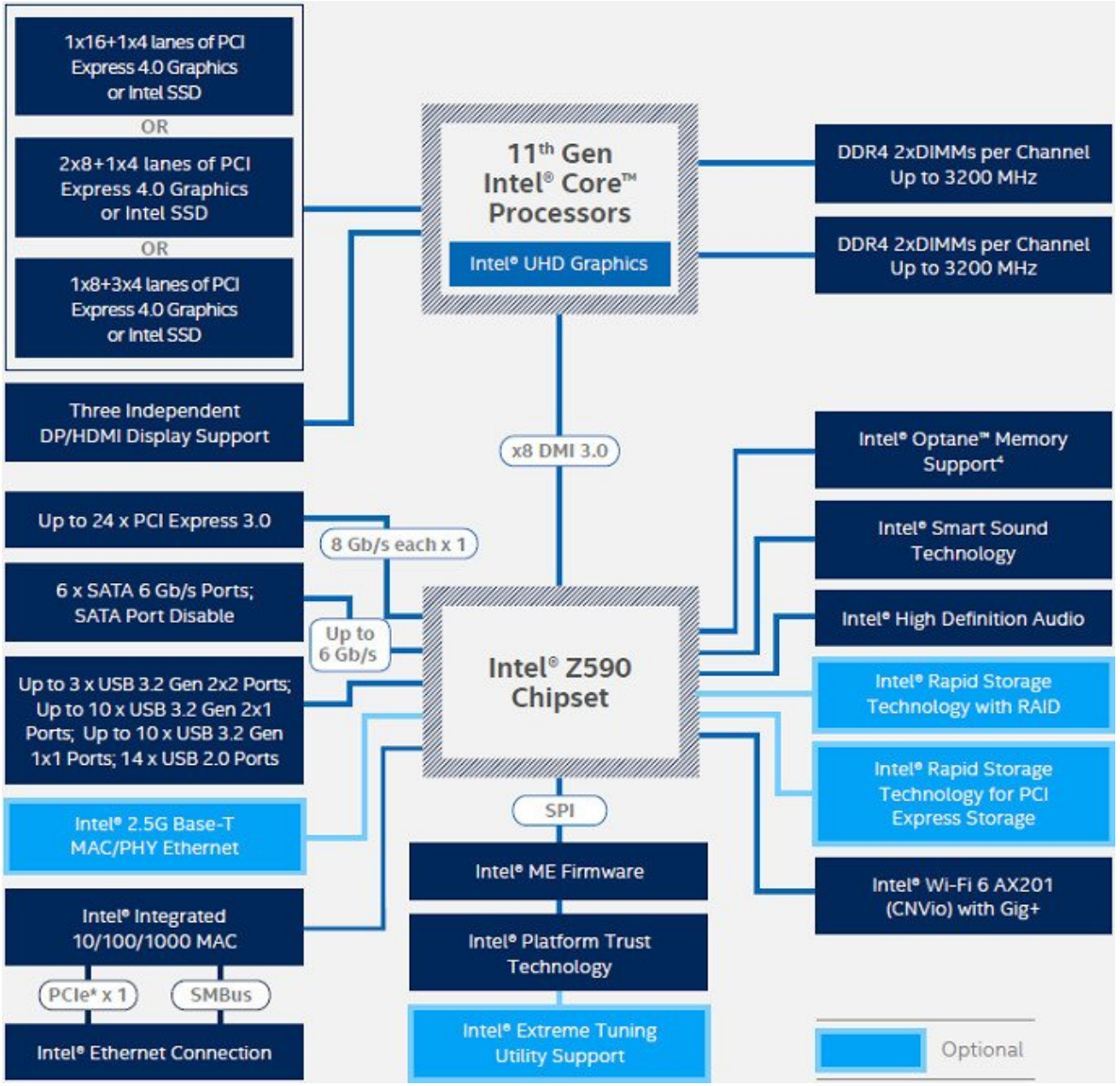
CS 202: Advanced Operating Systems

Overview

- I/O
  - How I/O devices talk to OS
- Storage devices
  - Hard disk drive: structure, latency, scheduling
- File system
  - Components
  - UFS

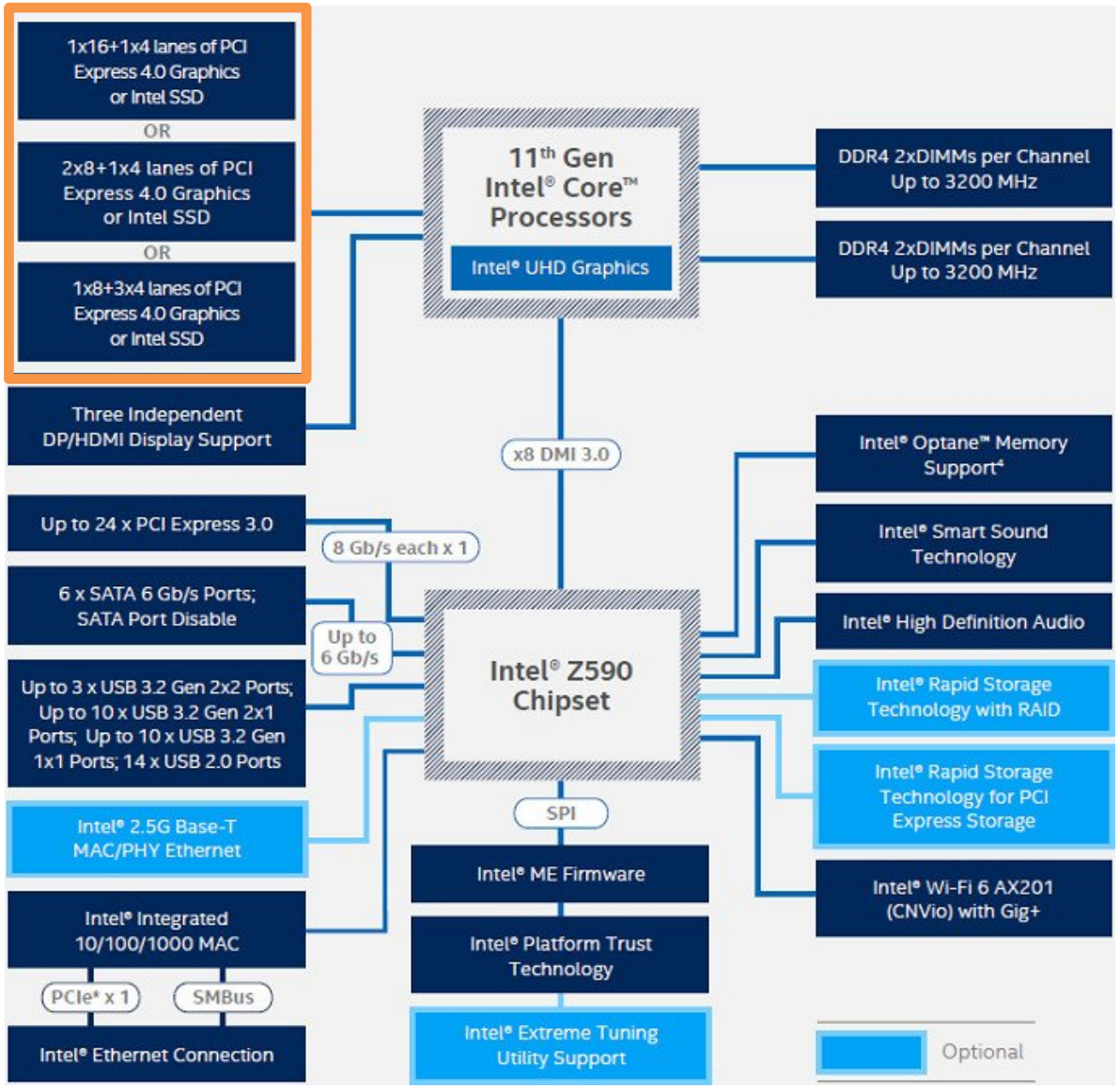
Bus Architecture

- Example: Intel Alder Lake

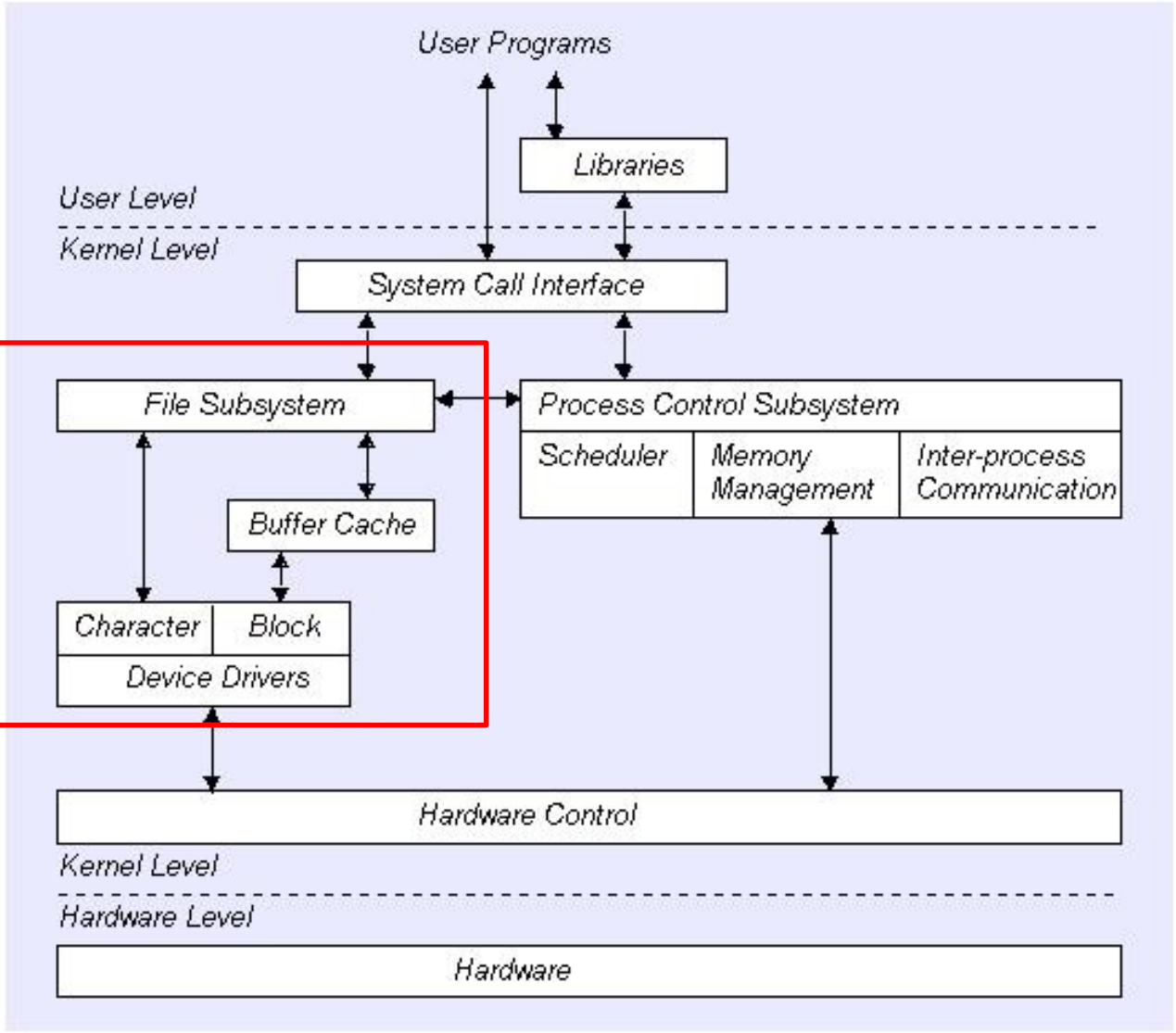


Bus Architecture

- Example: Intel Alder Lake



User Program and OS Interface



I/O Hardware

- Block devices
  - Stores information in fixed-size blocks, each one with its own address (512~32KB per block)
  - Possible to read or write each block independently of all the other ones
  - Disks, USB cameras, etc.
  - Commands include read(), write(), seek()

I/O Hardware

- Block devices
  - Stores information in fixed-size blocks, each one with its own address (512~32KB per block)
  - Possible to read or write each block independently of all the other ones
  - Disks, USB cameras, etc.
  - Commands include read(), write(), seek()
- Character devices
  - Delivers or accepts a stream of characters
  - Not addressable and no seek operation
  - Printers, networks, mice, keyboards, etc.
  - Commands include get(), put()

I/O device notifying the OS: Polling vs. Interrupts

- Polled I/O
  - CPU periodically checks ("polls") devices if need attention
  - Advantages
    - Simple; software is in control
    - Efficient if CPU finds a device to be ready soon (e.g., very short I/O operations)
  - Disadvantages
    - Wasting many cycles on polling; inefficient in non-trivial system (high CPU utilization)

I/O device notifying the OS: Polling vs. Interrupts

- Polled I/O
  - CPU periodically checks ("polls") devices if need attention
  - Advantages
    - Simple; software is in control
    - Efficient if CPU finds a device to be ready soon (e.g., very short I/O operations)
  - Disadvantages
    - Wasting many cycles on polling; inefficient in non-trivial system (high CPU utilization)

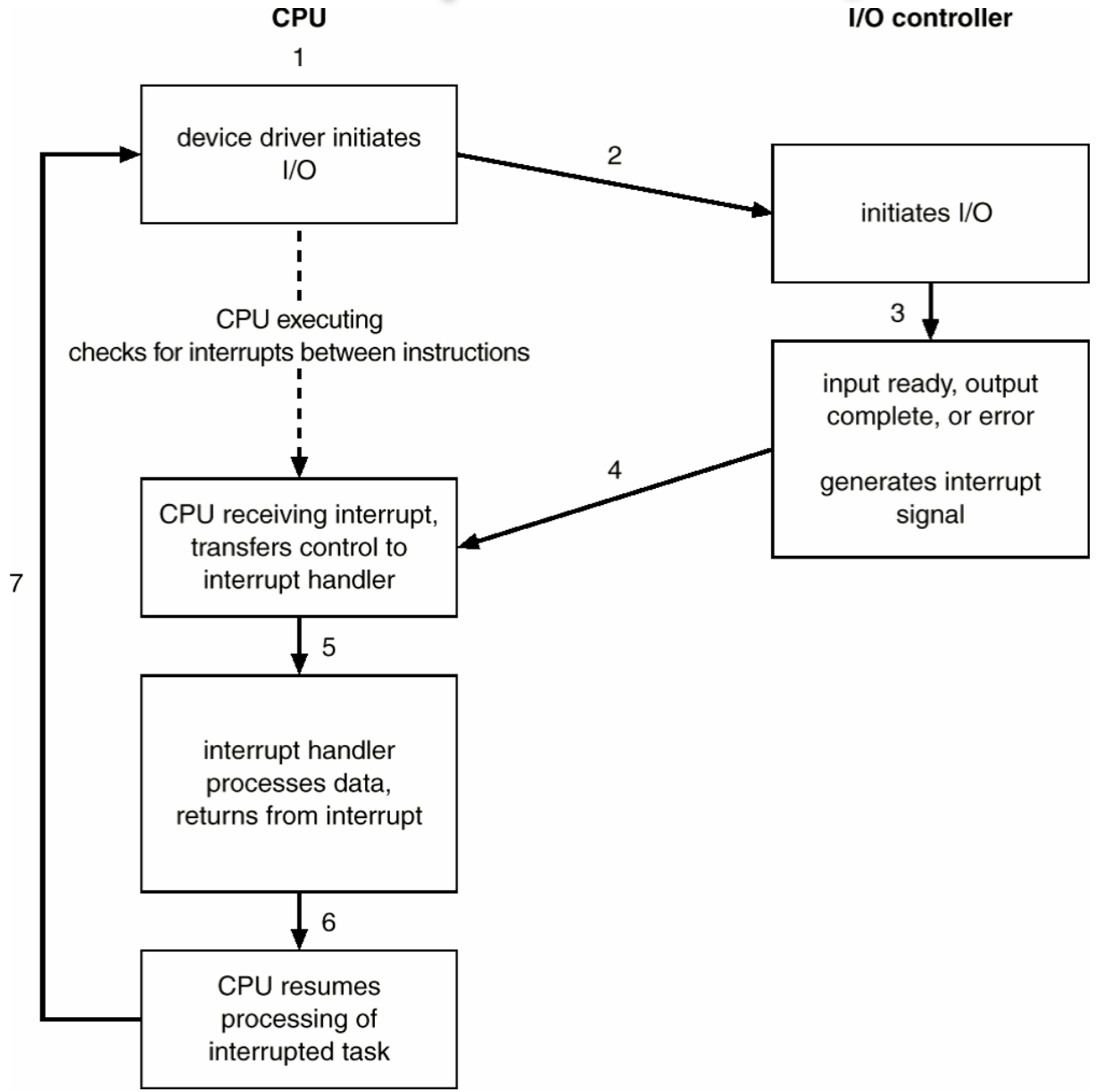
I/O device notifying the OS: Polling vs. Interrupts

- Polled I/O
  - CPU periodically checks ("polls") devices if need attention
  - Advantages
    - Simple; software is in control
    - Efficient if CPU finds a device to be ready soon (e.g., very short I/O operations)
  - Disadvantages
    - Wasting many cycles on polling; inefficient in non-trivial system (high CPU utilization)

I/O device notifying the OS: Polling vs. Interrupts

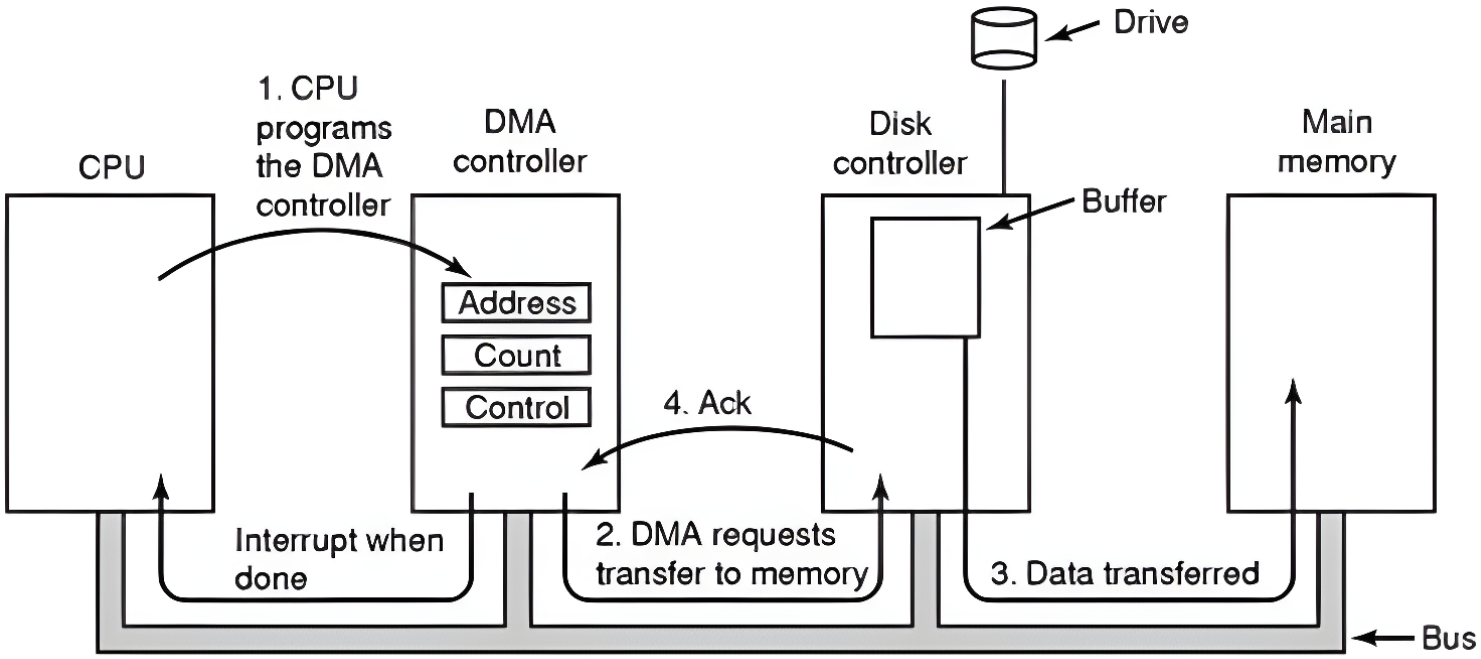
- Polled I/O
  - CPU periodically checks ("polls") devices if need attention
  - Advantages
    - Simple; software is in control
    - Efficient if CPU finds a device to be ready soon (e.g., very short I/O operations)
  - Disadvantages
    - Wasting many cycles on polling; inefficient in non-trivial system (high CPU utilization)
- Interrupt-driven I/O
  - I/O devices request interrupt when need attention
  - Advantages
    - CPU only attends to device when necessary
    - Handle infrequent or unpredictable events very well
    - More efficient than polling in general
  - Disadvantages
    - Overheads (mode switch, context switch, interrupt handling)
- What would a high-bandwidth network adapter do?

Interrupt-driven I/O

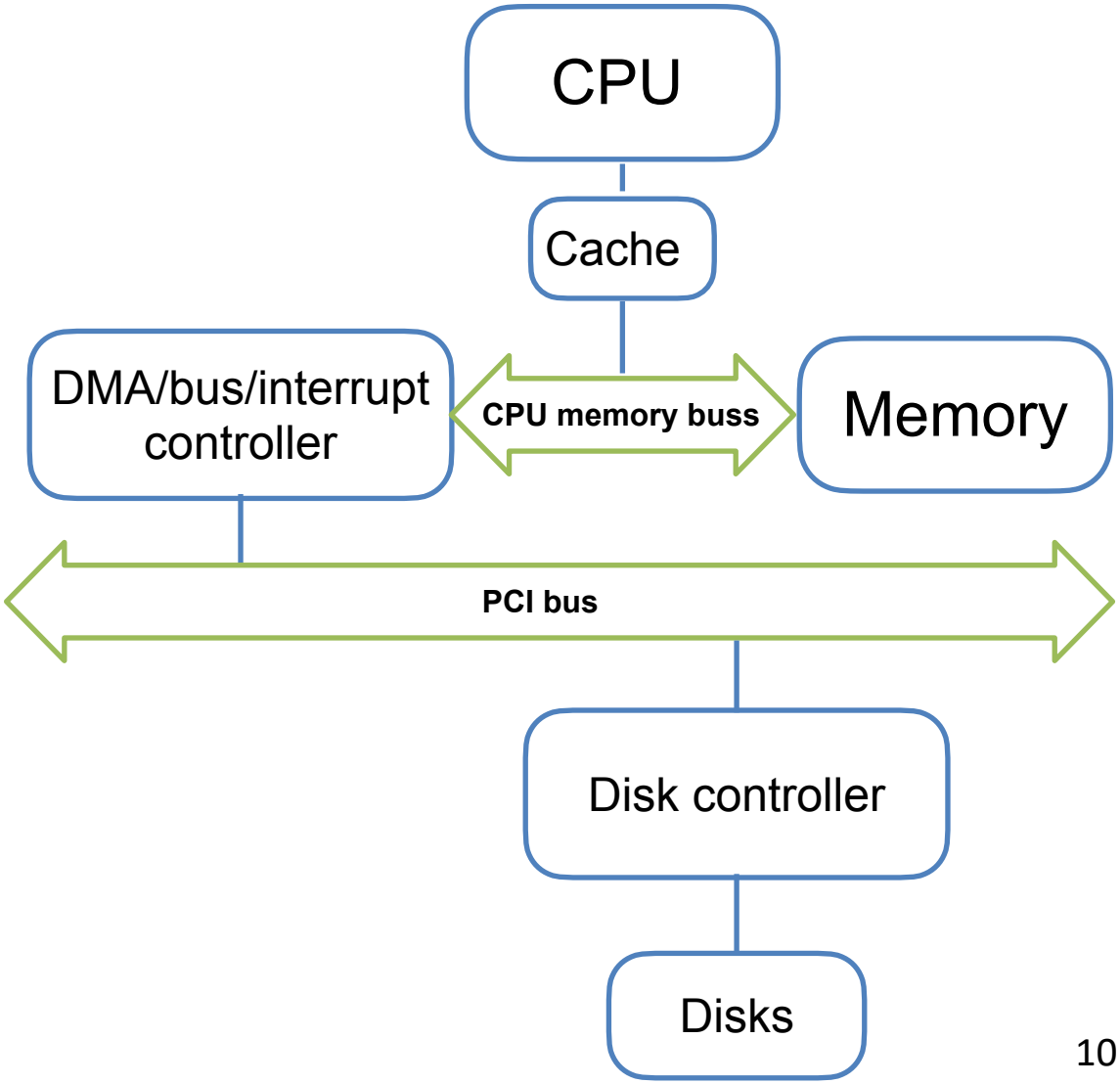


Direct Memory Access

- DMA
  - Bypasses CPU to transfer data directly between I/O device and memory
  - Used to avoid programmed I/O for large data movement
  - Require DMA controller

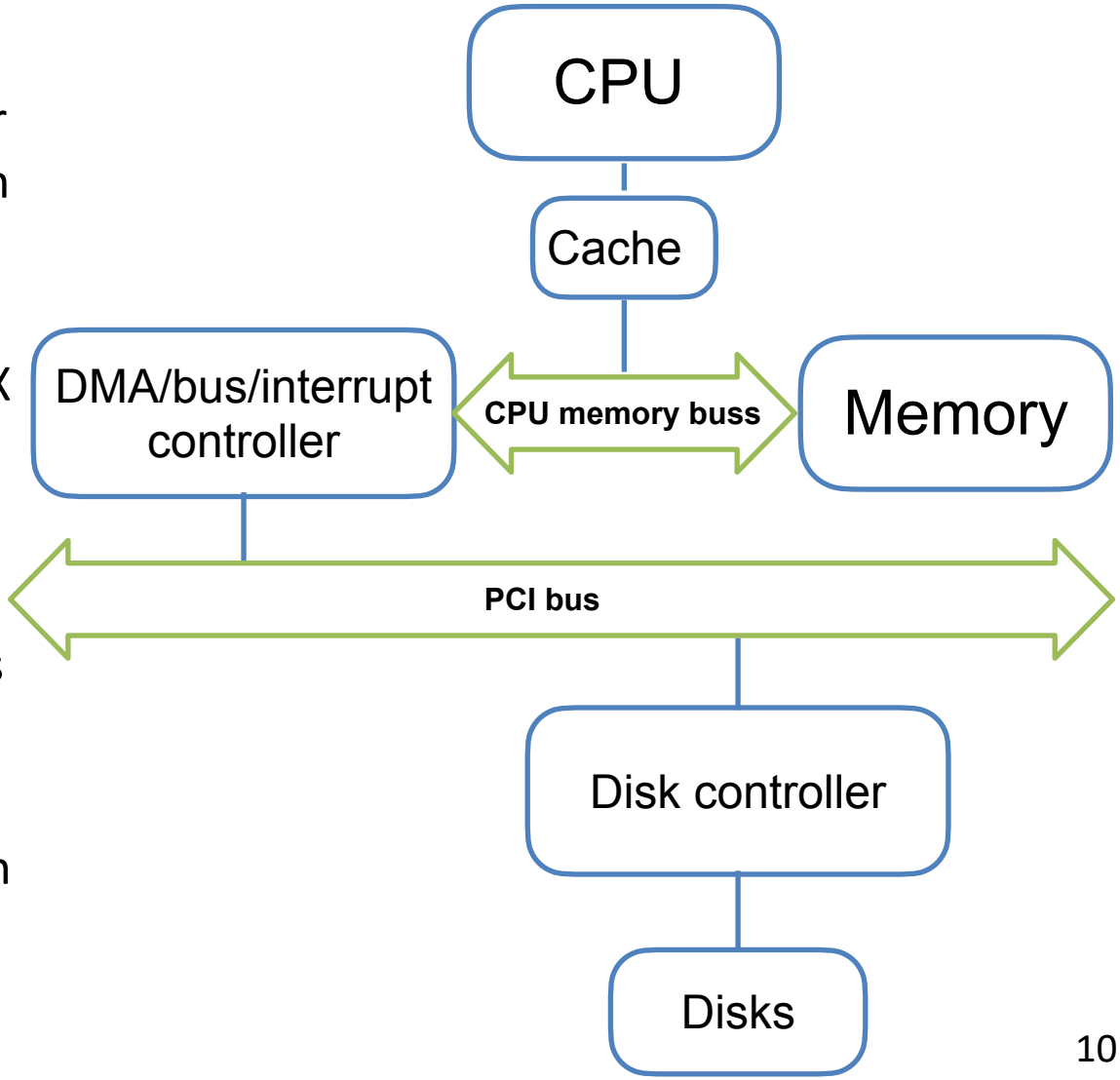


Direct Memory Access



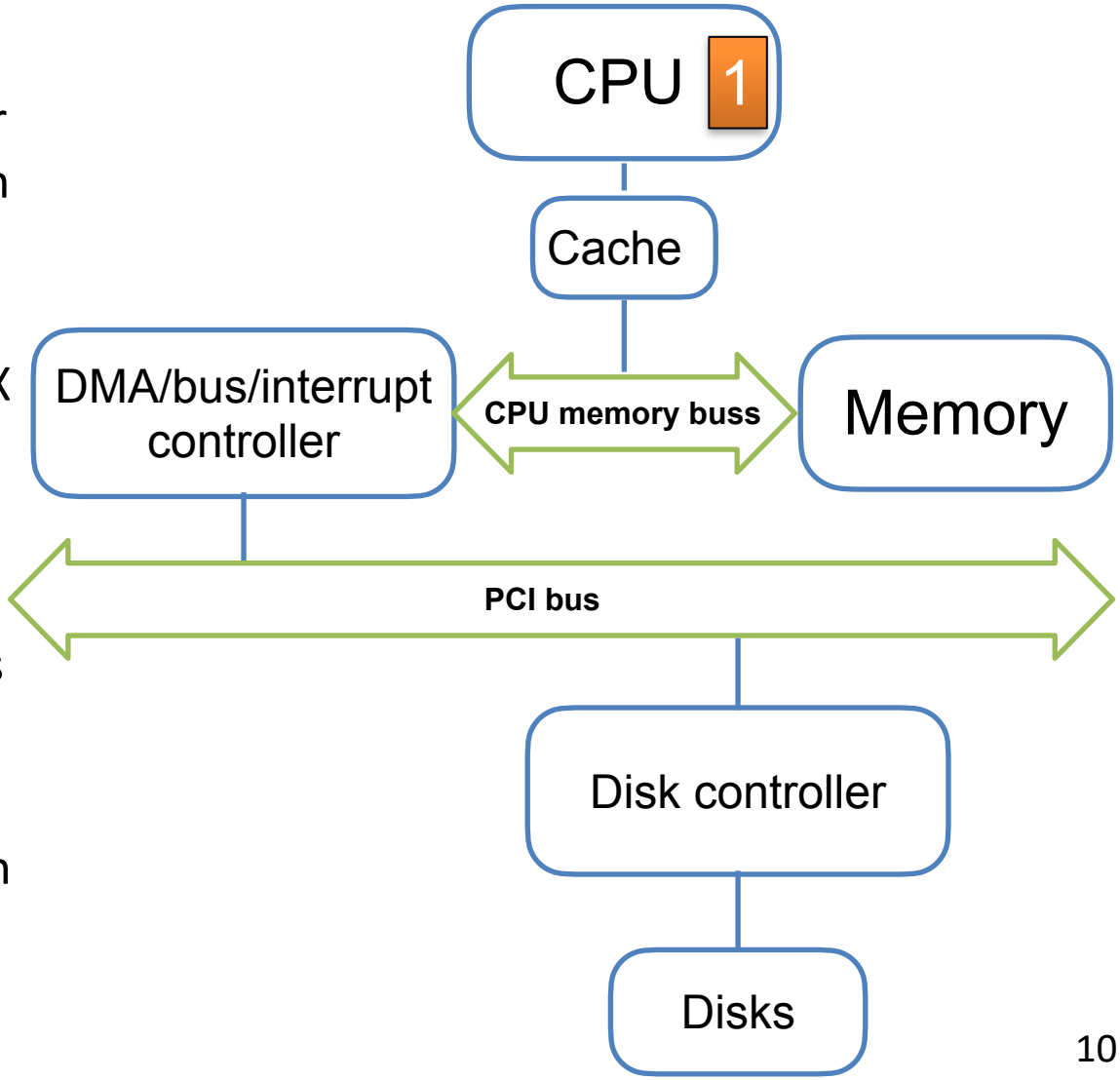
Direct Memory Access

1. Device driver is told to transfer disk data to buffer at address X in memory
2. Device driver tells disk controller to transfer N bytes to X
3 & 4. Disk controller initiates DMA transfer, and sends each byte to DMA controller
5. DMA controller transfers bytes to X, decreasing N until N=0
6. When N=0, DMA interrupts CPU to signal transfer completion



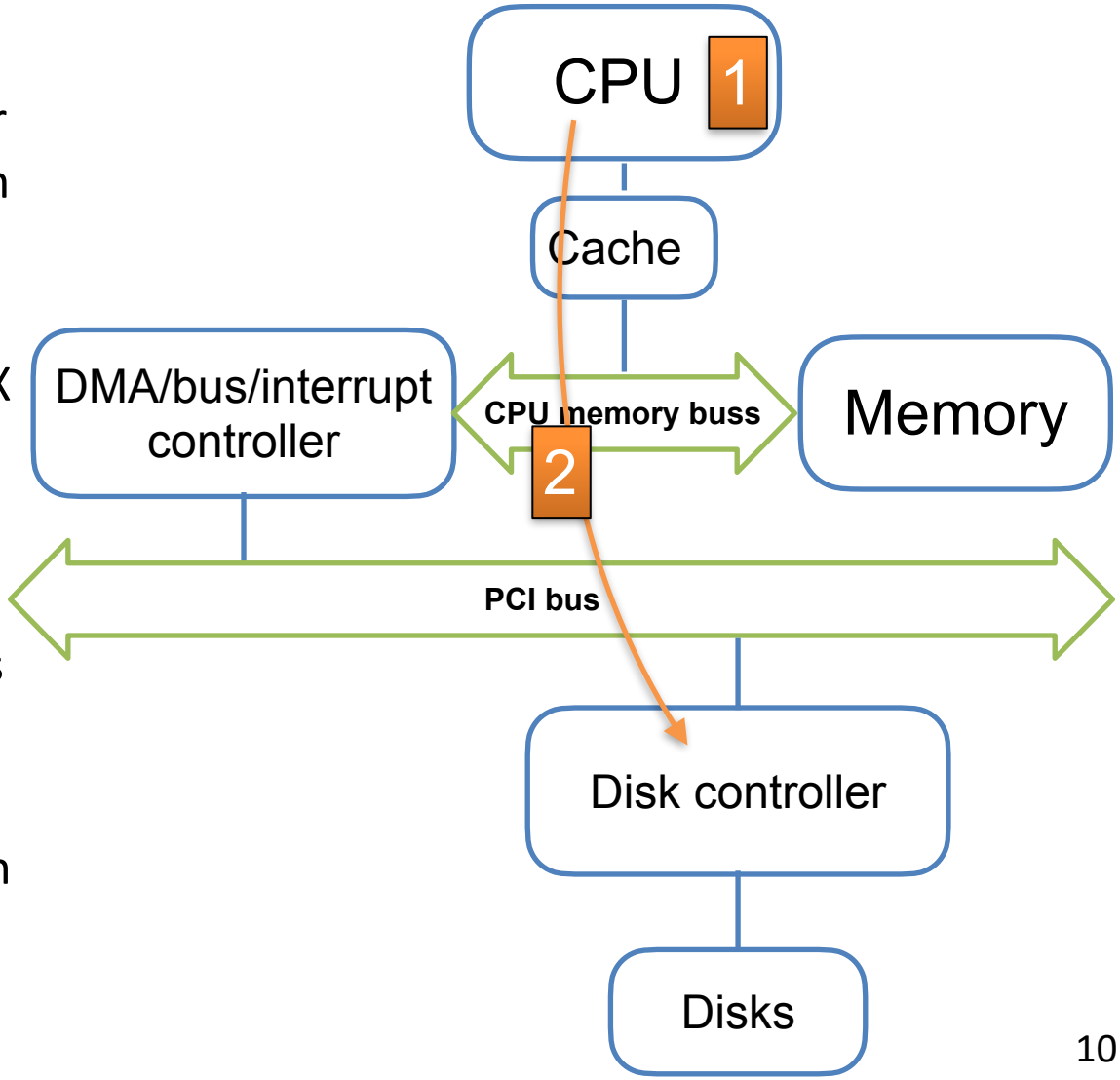
Direct Memory Access

1. Device driver is told to transfer disk data to buffer at address X in memory
2. Device driver tells disk controller to transfer N bytes to X
3 & 4. Disk controller initiates DMA transfer, and sends each byte to DMA controller
5. DMA controller transfers bytes to X, decreasing N until N=0
6. When N=0, DMA interrupts CPU to signal transfer completion



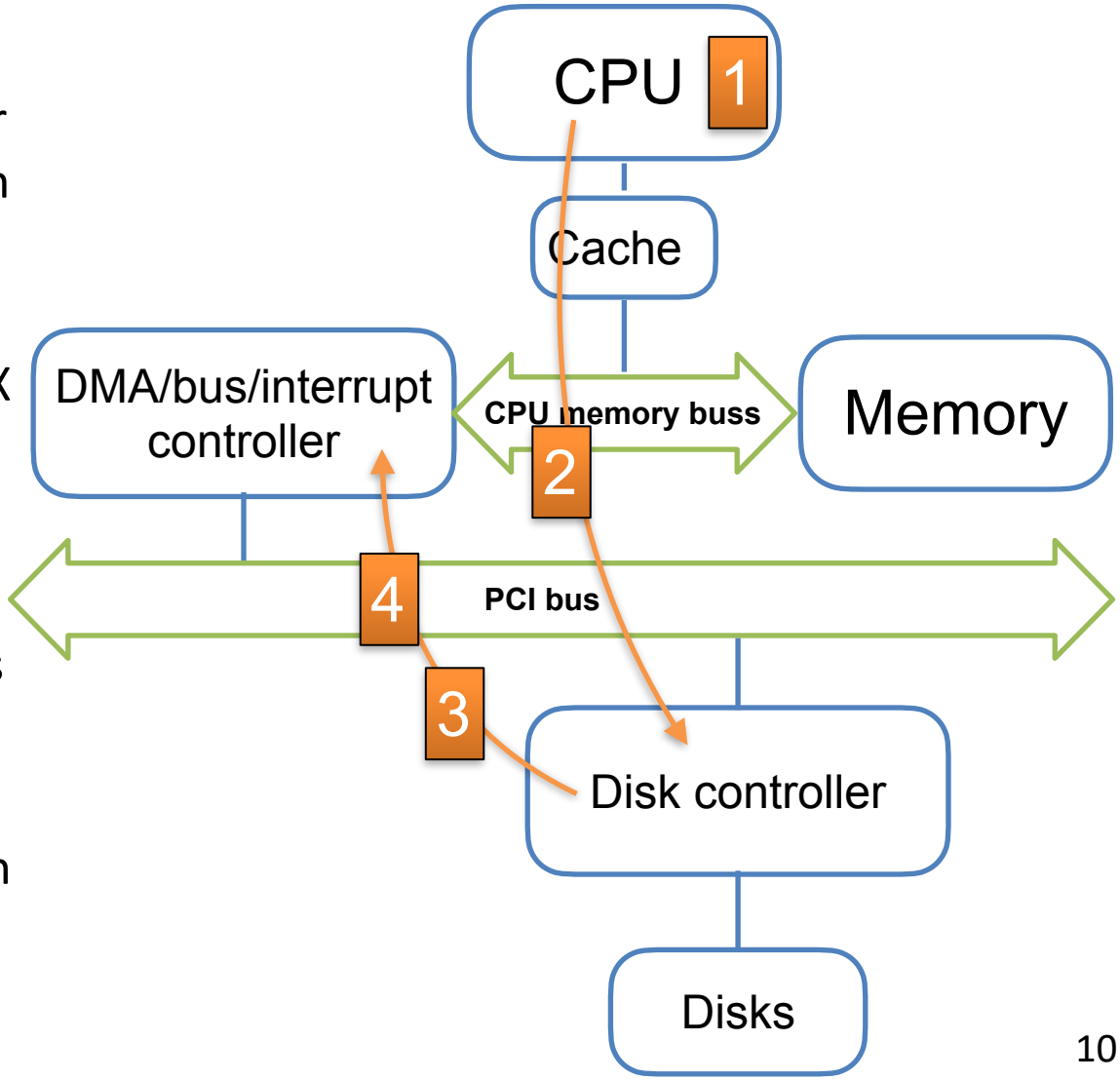
Direct Memory Access

1. Device driver is told to transfer disk data to buffer at address X in memory
2. Device driver tells disk controller to transfer N bytes to X
3 & 4. Disk controller initiates DMA transfer, and sends each byte to DMA controller
5. DMA controller transfers bytes to X, decreasing N until N=0
6. When N=0, DMA interrupts CPU to signal transfer completion



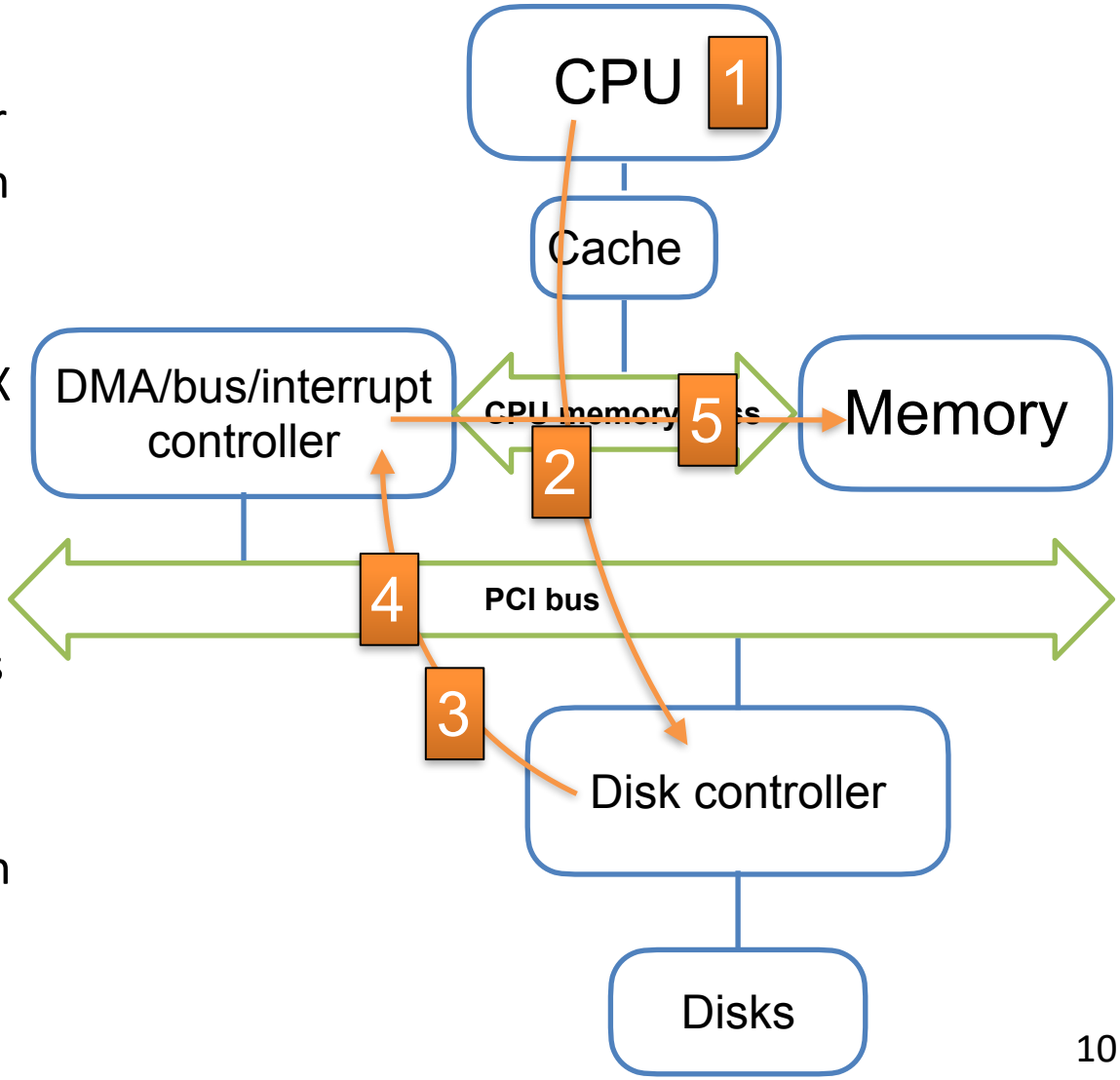
Direct Memory Access

1. Device driver is told to transfer disk data to buffer at address X in memory
2. Device driver tells disk controller to transfer N bytes to X
3 & 4. Disk controller initiates DMA transfer, and sends each byte to DMA controller
5. DMA controller transfers bytes to X, decreasing N until N=0
6. When N=0, DMA interrupts CPU to signal transfer completion



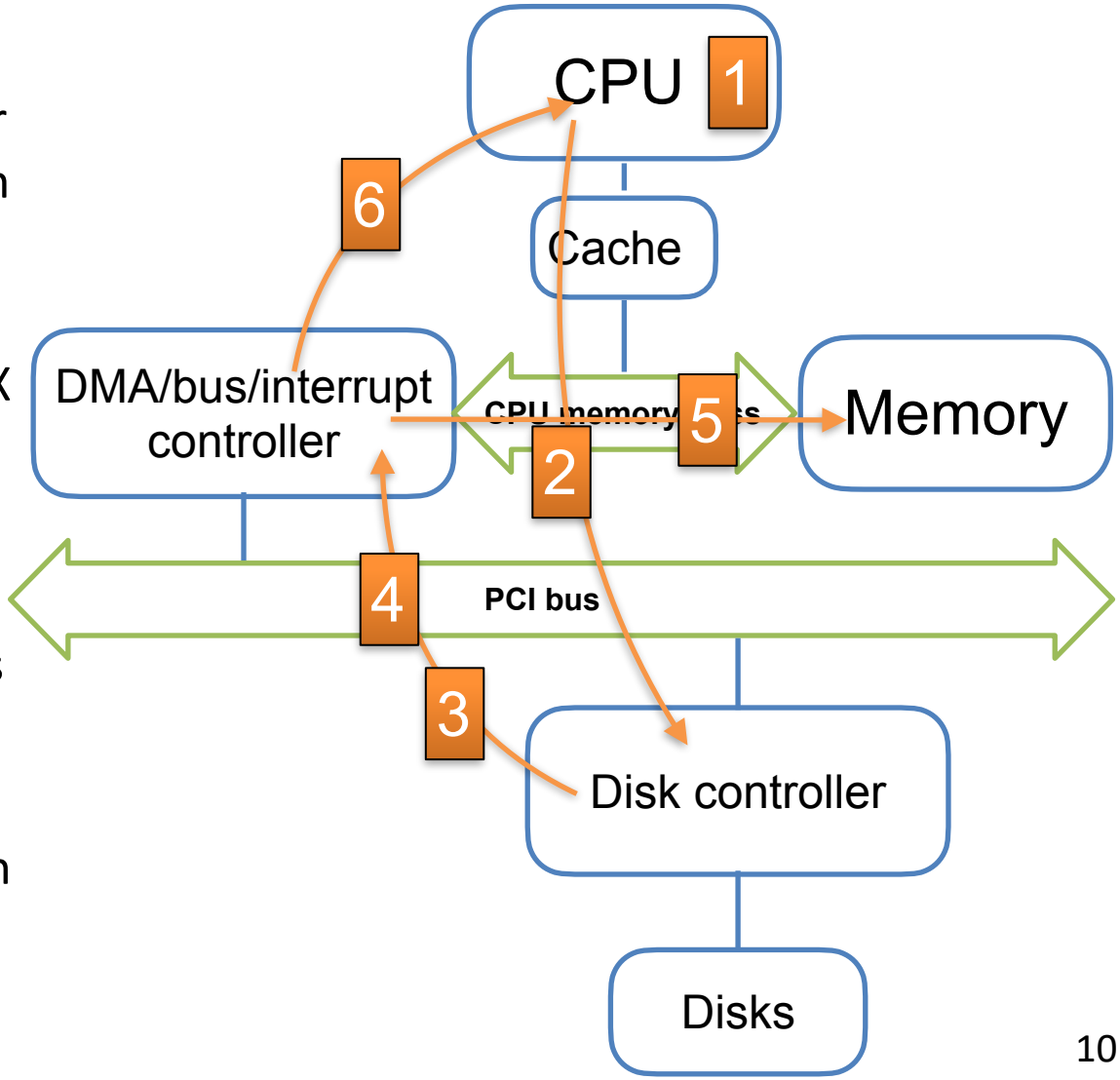
Direct Memory Access

1. Device driver is told to transfer disk data to buffer at address X in memory
2. Device driver tells disk controller to transfer N bytes to X
3 & 4. Disk controller initiates DMA transfer, and sends each byte to DMA controller
5. DMA controller transfers bytes to X, decreasing N until N=0
6. When N=0, DMA interrupts CPU to signal transfer completion



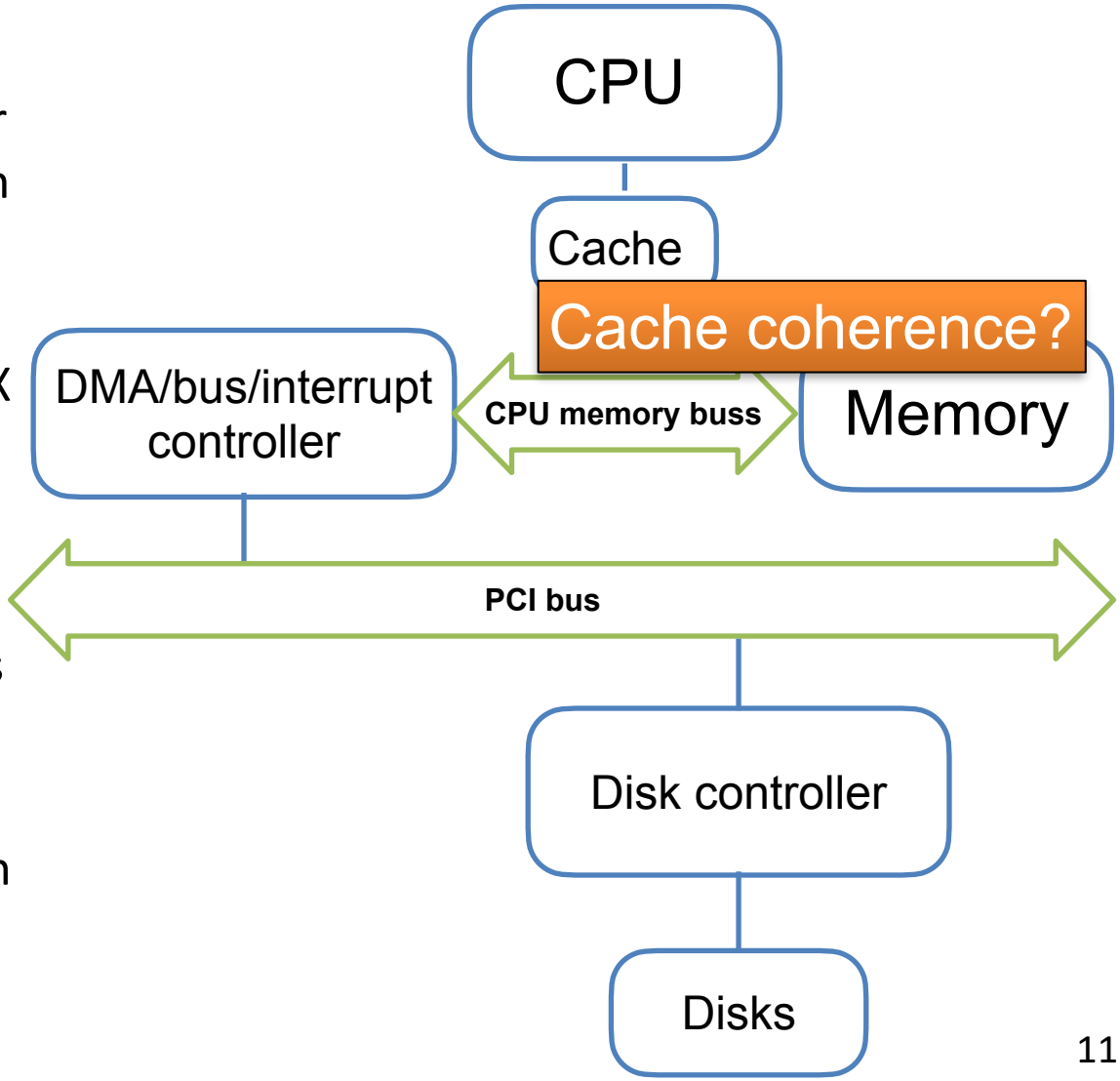
Direct Memory Access

1. Device driver is told to transfer disk data to buffer at address X in memory
2. Device driver tells disk controller to transfer N bytes to X
3 & 4. Disk controller initiates DMA transfer, and sends each byte to DMA controller
5. DMA controller transfers bytes to X, decreasing N until N=0
6. When N=0, DMA interrupts CPU to signal transfer completion



Direct Memory Access

1. Device driver is told to transfer disk data to buffer at address X in memory
2. Device driver tells disk controller to transfer N bytes to X
3 & 4. Disk controller initiates DMA transfer, and sends each byte to DMA controller
5. DMA controller transfers bytes to X, decreasing N until N=0
6. When N=0, DMA interrupts CPU to signal transfer completion



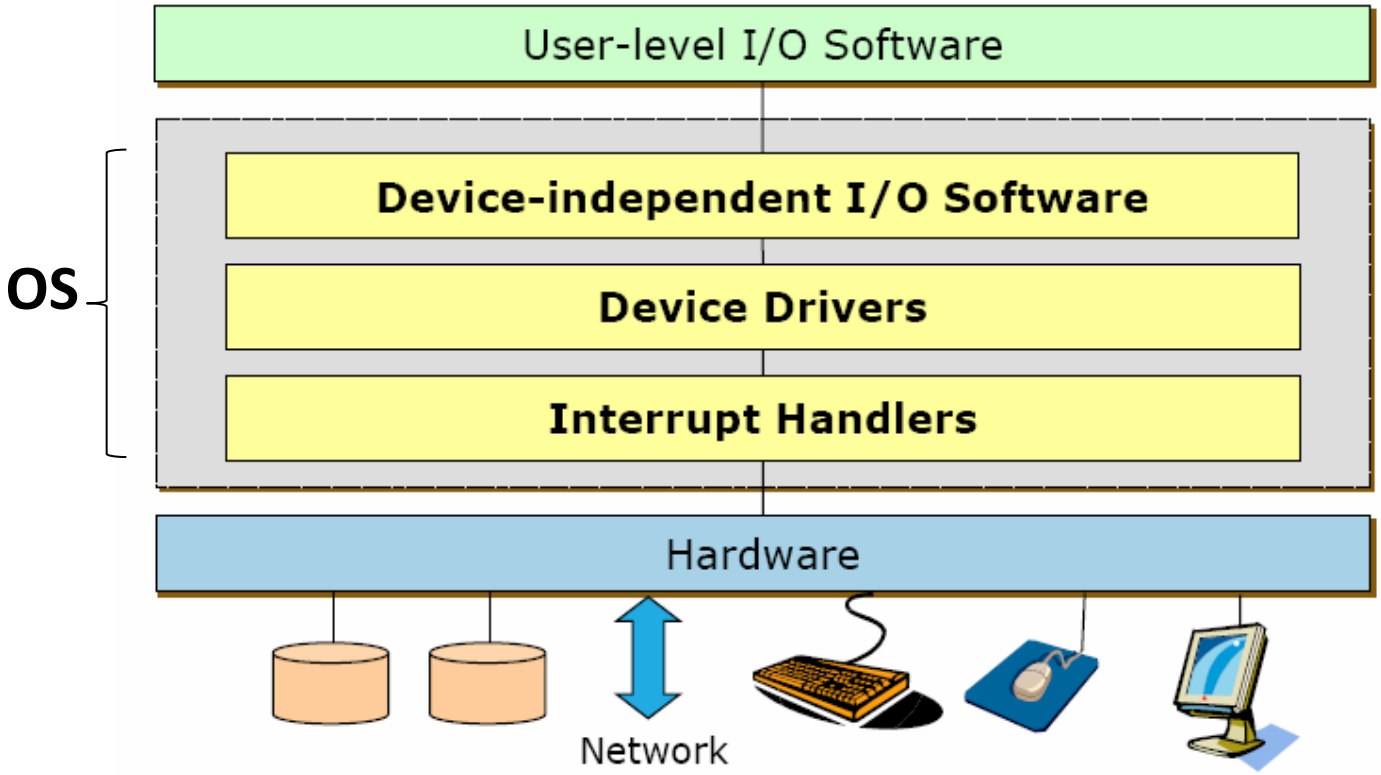
Discussion Questions

Which of the following best explains why DMA improves I/O performance?

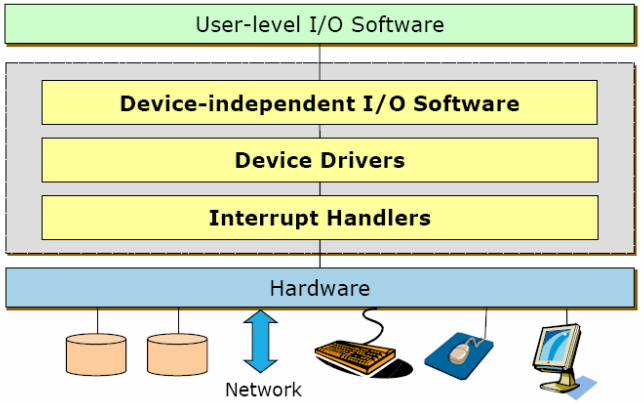
A. It increases the cache hit rate for I/O operations.
B. It prefetches memory before device access to reduce latency.
C. It bypasses the CPU, allowing direct memory-device transfer, thus reducing CPU overhead.
D. It allows user applications to directly manipulate device registers.

I/O Software Layers

- Gives a uniform I/O interface to applications for different types of devices

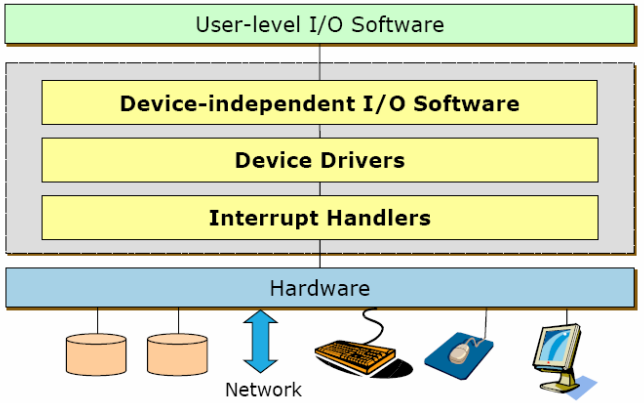


I/O Software Layers



- Device-Independent I/O Interface
  - Uniform interface for device drivers
  - In UNIX, devices are modeled as files (e.g., /dev)
    - Accessed via system calls such as open(), read(), write(), ioctl(), etc.

I/O Software Layers



- Device-Independent I/O Interface
  - Uniform interface for device drivers
  - In UNIX, devices are modeled as files (e.g., /dev)
    - Accessed via system calls such as open(), read(), write(), ioctl(), etc.
    - Start I/O to device, and may put thread to sleep until finished
- Device drivers
  - Device-specific code to control each I/O device
  - Interacts with device-independent layer and interrupt handlers
  - Hides differences among I/O controllers from kernel
    - E.g., device drivers for SSD, HDD, etc.

Improving I/O Performance

- I/O: a major factor in system performance
  - Demands CPU to execute device driver, kernel I/O code
  - Context switches due to interrupts
  - Data copying

Improving I/O Performance

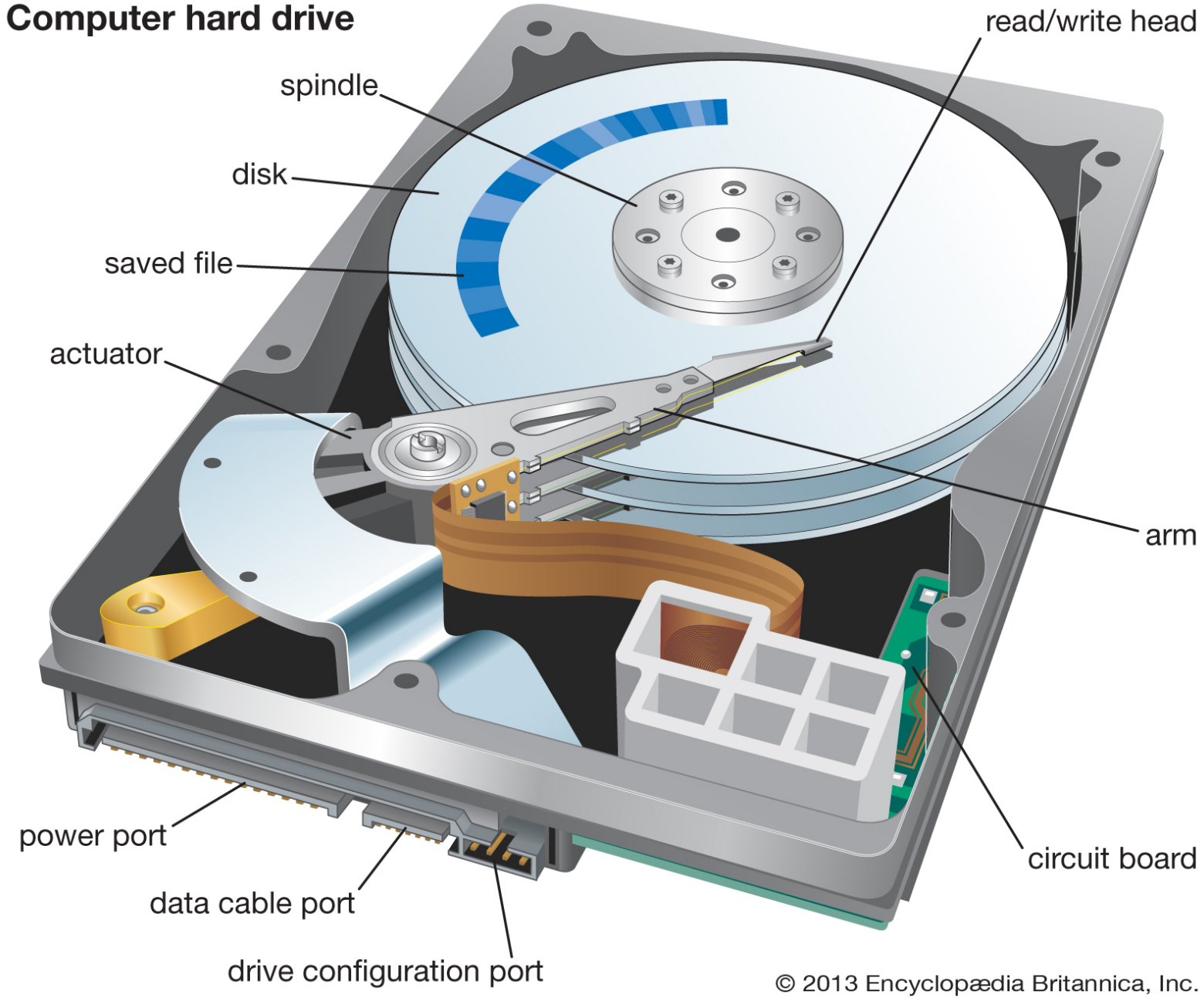
- I/O: a major factor in system performance
  - Demands CPU to execute device driver, kernel I/O code
  - Context switches due to interrupts
  - Data copying
- General guidelines
  - Use DMA
  - Reduce interrupts by using large transfers, smart controllers, polling
  - Move processing primitives into hardware; concurrent operations of CPU and device controllers
  - Reduce number of context switches
  - Reduce data copying
  - Balance CPU, memory, bus, and I/O performance for highest throughput

Storage Devices

(some slides credit to Christo Wilson & Ariel J. Frank)

Storage Devices

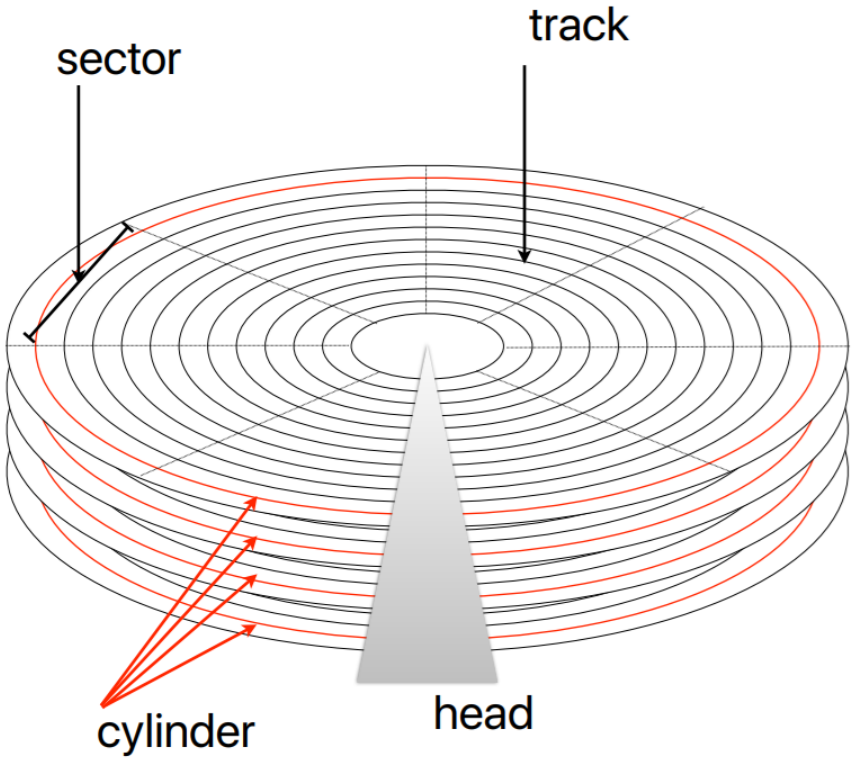
- Magnetic disks
  - Large capacity at low cost
  - Slow (better) performance for random (sequential) access
- Flash memory (e.g., SSD)
  - Capacity at more cost (10-20x disk)
  - Good performance on reads; worse for random writes
  - Erase operations due to writes cause a small amount of damage to the memory cell
  - Wear out issue: writing flash memory too often would wear it out (losing bits)



© 2013 Encyclopædia Britannica, Inc.

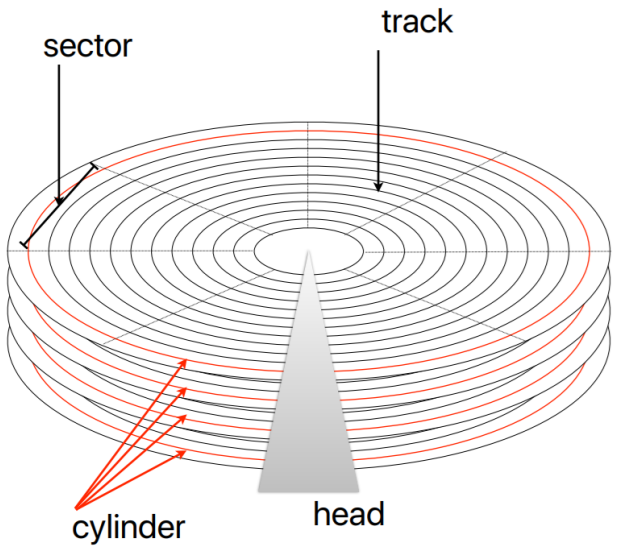
Hard Disk Drive

- Sectors
  - Typically 512 or 4096 bytes
  - Individual sector writes are atomic
  - Multiple sector writes may be interrupted
- Sectors are arranged into tracks
- Tracks are arranged in circles on platters
- Cylinder is a particular track on multiple platters
- Disk may have multiple platters
- Driver motor spins the platter at a constant speed
  - RPM: revolutions per minute



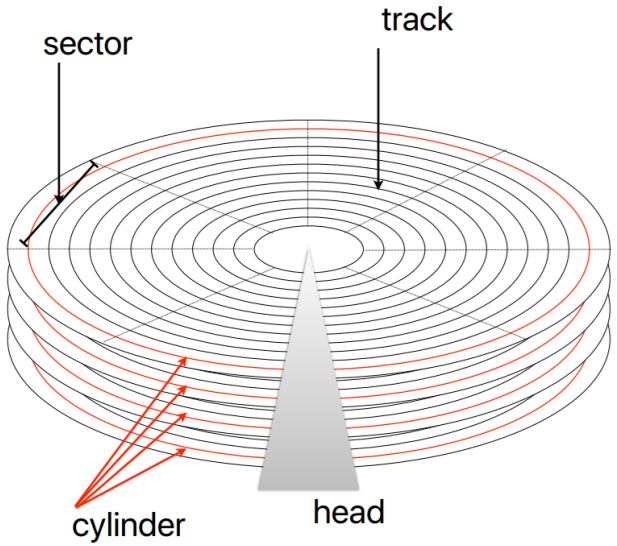
Hard Disk Drive

- Read/write data is a three-stage process:
  - Seek time: position the head/arm over the proper track (into proper cylinder)
  - Rotational latency: wait for the desired sector to rotate under the read/write head
  - Transfer time: transfer a block of bits (sector) under the read-write head



Hard Disk Drive

- Read/write data is a three-stage process:
  - Seek time: position the head/arm over the proper track (into proper cylinder)
  - Rotational latency: wait for the desired sector to rotate under the read/write head
  - Transfer time: transfer a block of bits (sector) under the read-write head

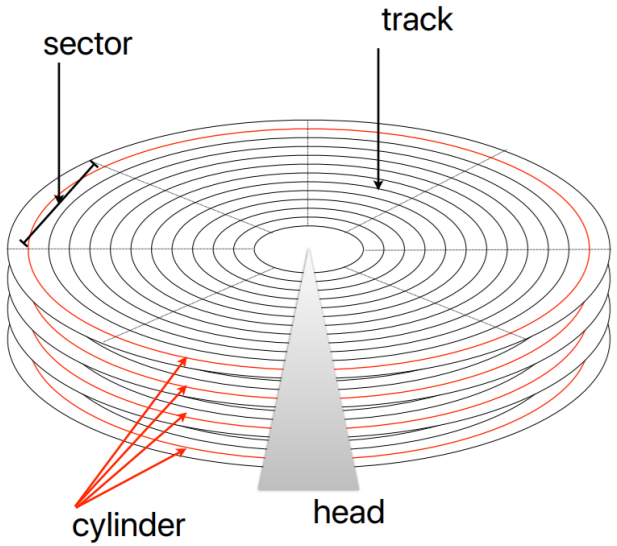


- Disk I/O time = Seek Time + Rotation Time + Transfer Time
  - (Note: end-to-end latency = queuing time + controller time + disk time)



Hard Disk Drive

- Read/write data is a three-stage process:
  - Seek time: position the head/arm over the proper track (into proper cylinder)
  - Rotational latency: wait for the desired sector to rotate under the read/write head
  - Transfer time: transfer a block of bits (sector) under the read-write head



- Disk I/O time = Seek Time + Rotation Time + Transfer Time
  - (Note: end-to-end latency = queuing time + controller time + disk time)



- Highest Disk Bandwidth:
  - The maximum amount of data that can be transferred to and from a disk over a specified period of time
  - Typically MB/s or GB/s

How To Calculate Disk Latency

Seagate

	Cheetah 15K.5	Barracuda
Capacity	300 GB	1 TB
RPM	15000	7200
Avg. Seek	4 ms	9 ms
Max Transfer	125 MB/s	105 MB/s

How To Calculate Disk Latency

Seagate

	Cheetah 15K.5	Barracuda
Capacity	300 GB	1 TB
RPM	15000	7200
Avg. Seek	4 ms	9 ms
Max Transfer	125 MB/s	105 MB/s

Disk I/O time

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

- Assume we are transferring 4096 bytes (1 block)
- Ignore the controller time & queueing time
- ***Rotational latency is half the time it takes the disk to make a complete revolution***

How To Calculate Disk Latency

Seagate

	Cheetah 15K.5	Barracuda
Capacity	300 GB	1 TB
RPM	15000	7200
Avg. Seek	4 ms	9 ms
Max Transfer	125 MB/s	105 MB/s

Disk I/O time

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

- Assume we are transferring 4096 bytes (1 block)
- Ignore the controller time & queueing time
- ***Rotational latency is half the time it takes the disk to make a complete revolution***

Cheetah

$$T_{I/O} = 4\ ms + (1\ /\ (15000\ RPM\ /\ 60\ s/M\ /\ 1000\ ms/s))\ /\ 2 + (4096\ B\ /\ (125\ MB/s\ *\ 2^{20}\ B/MB\ /\ 1000\ ms/s)$$

$$= 4\ ms + 2ms + 0.03125\ ms \approx 6\ ms$$

How To Calculate Disk Latency

Seagate

	Cheetah 15K.5	Barracuda
Capacity	300 GB	1 TB
RPM	15000	7200
Avg. Seek	4 ms	9 ms
Max Transfer	125 MB/s	105 MB/s

Disk I/O time

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

- Assume we are transferring 4096 bytes (1 block)
- Ignore the controller time & queueing time
- ***Rotational latency is half the time it takes the disk to make a complete revolution***

Cheetah

$$T_{I/O} = 4\ ms + (1 / (15000\ RPM / 60\ s/M / 1000\ ms/s)) / 2 + (4096\ B / (125\ MB/s * 2^{20}\ B/MB / 1000\ ms/s)$$

$$= 4\ ms + 2ms + 0.03125\ ms \approx 6\ ms$$

Barracuda

$$T_{I/O} = 9\ ms + (1 / (7200\ RPM / 60\ s/M / 1000\ ms/s)) / 2 + (4096\ B / (105\ MB/s * 2^{20}\ B/MB / 1000\ ms/s)$$

$$= 9\ ms + 4.17\ ms + 0.0372\ ms \approx 13.2\ ms$$

UCR

Sequential vs. Random Access

- Assume we are transferring 125MB of data in a block of 4096 bytes

	Cheetah 15K.5
Capacity	300 GB
RPM	15000
Avg. Seek	4 ms
Max Transfer	125 MB/s

Sequential vs. Random Access

- Assume we are transferring 125MB of data in a block of 4096 bytes

- Sequential access

$T_{I/O}$ = *4 ms + (1 / (15000 RPM / 60 s/M / 1000 ms/s)) / 2*

+ (125MB / (125 MB/s / 1000 ms/s)

= 4 ms + 2ms + 1000 ms = 1006 ms

Throughput = 100 MB / 1.006 s ≈ *124.25 MB/s*

	Cheetah 15K.5
Capacity	300 GB
RPM	15000
Avg. Seek	4 ms
Max Transfer	125 MB/s

Sequential vs. Random Access

- Assume we are transferring 125MB of data in a block of 4096 bytes

	Cheetah 15K.5
Capacity	300 GB
RPM	15000
Avg. Seek	4 ms
Max Transfer	125 MB/s

- Sequential access

$T_{I/O}$ = *4 ms + (1 / (15000 RPM / 60 s/M / 1000 ms/s)) / 2*
+ (125MB / (125 MB/s / 1000 ms/s)

= 4 ms + 2ms + 1000 ms = 1006 ms

Throughput = 100 MB / 1.006 s ≈ *124.25 MB/s*

- Random access

For each 4K block, $T_{I/O=4KB}$ ≈ 6 ms

*Delay for 100 MB ≈ 6 ms * (100 MB * 2^{20} B/MB / 4096 B) = 153600 ms*

Throughput = 100 MB / 153.6 s ≈ *0.65 MB/s*

Sequential vs. Random Access

- Assume we are transferring 125MB of data in a block of 4096 bytes

	Cheetah 15K.5
Capacity	300 GB
RPM	15000
Avg. Seek	4 ms
Max Transfer	125 MB/s

- Sequential access

$T_{I/O}$ = *4 ms + (1 / (15000 RPM / 60 s/M / 1000 ms/s)) / 2*
+ (125MB / (125 MB/s / 1000 ms/s)

= 4 ms + 2ms + 1000 ms = 1006 ms

Throughput = 100 MB / 1.006 s ≈ *124.25 MB/s*

- Random access

Random I/O results in very poor disk performance!

For each 4K block, $T_{I/O=4KB}$ ≈ *6 ms*

*Delay for 100 MB ≈ 6 ms * (100 MB * 2^{20} B/MB / 4096 B) = 153600 ms*

Throughput = 100 MB / 153.6 s ≈ *0.65 MB/s*

Discussion Questions

When analyzing disk access times, why is random access performance significantly worse than sequential access?

A. Random access benefits less from DMA acceleration.
B. File systems ignore rotational latency in random access.
C. Each random access may incur a full seek + rotational delay, dramatically increasing per-block overhead.
D. Random access causes bus contention and packet loss in the PCIe controller.

Disk scheduling

- The operating system is responsible for using hardware efficiently — for the disk drives, this means having a fast access time and disk bandwidth
- Access time has two major components:
  - Seek time is the time for the disk are to move the heads to the cylinder containing the desired sector
  - Rotational latency is the additional time waiting for the disk to rotate the desired sector to the disk head.
- Minimize seek time ~= seek distance
- Disk bandwidth is the total number of bytes transferred, divided by the total time between the first request for service and the completion

Disk scheduling

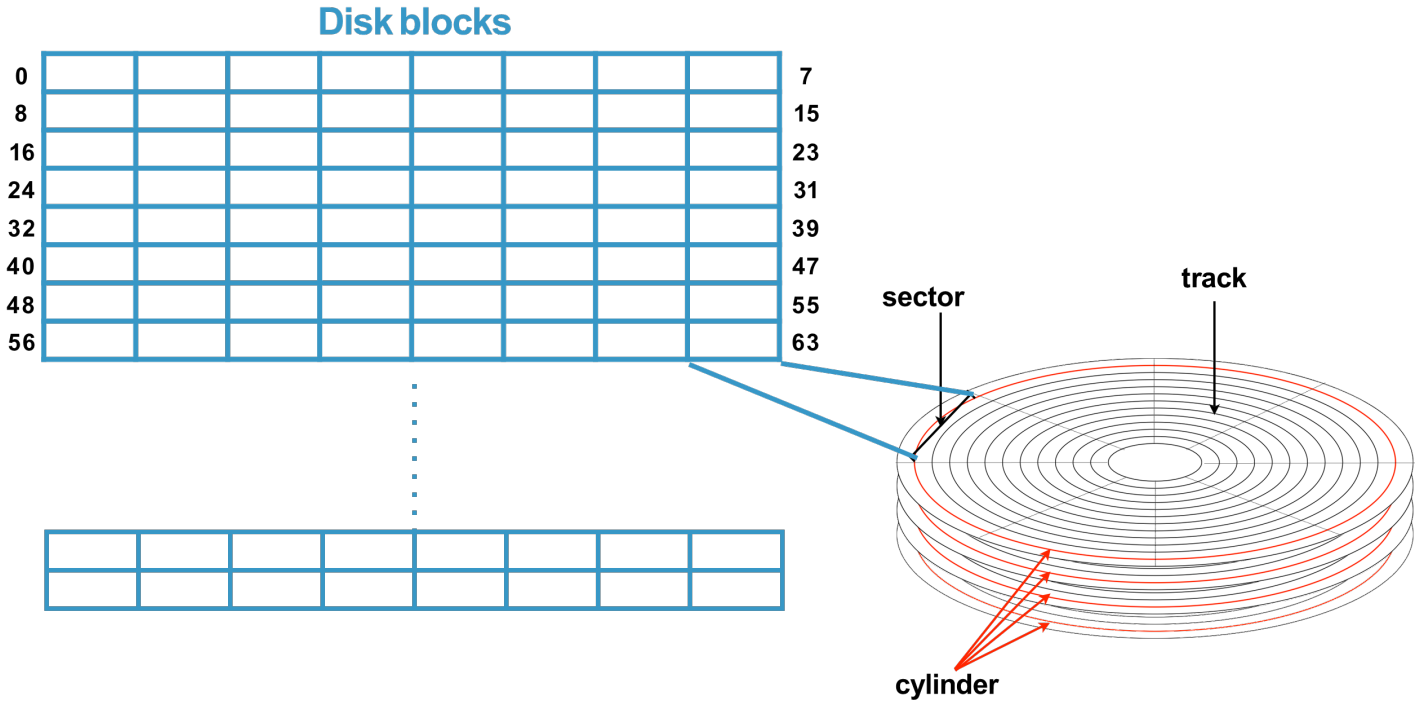
- Many sources of disk I/O request:
  - OS
  - System processes
  - User processes
- I/O request includes input/output mode, disk address, memory address, number of sectors to transfer
- OS maintains queue of requests, per disk or device
- Idle disk can immediately work on I/O request, busy disk means work must queue:
  - Optimization algorithms only make sense when a queue exists

Disk scheduling

- Disk drives are addressed as large 1-dimensional arrays of logical blocks, where the logical block is the smallest unit of transfer
- The 1-dimensional array of logical blocks is mapped into the sectors of the disk sequentially:
  - Sector 0 is the first sector of the first track on the outermost cylinder
  - Mapping proceeds in order through that track, then the rest of the tracks in that cylinder, and then through the rest of the cylinders from outermost to innermost.

Logical Block Addressing

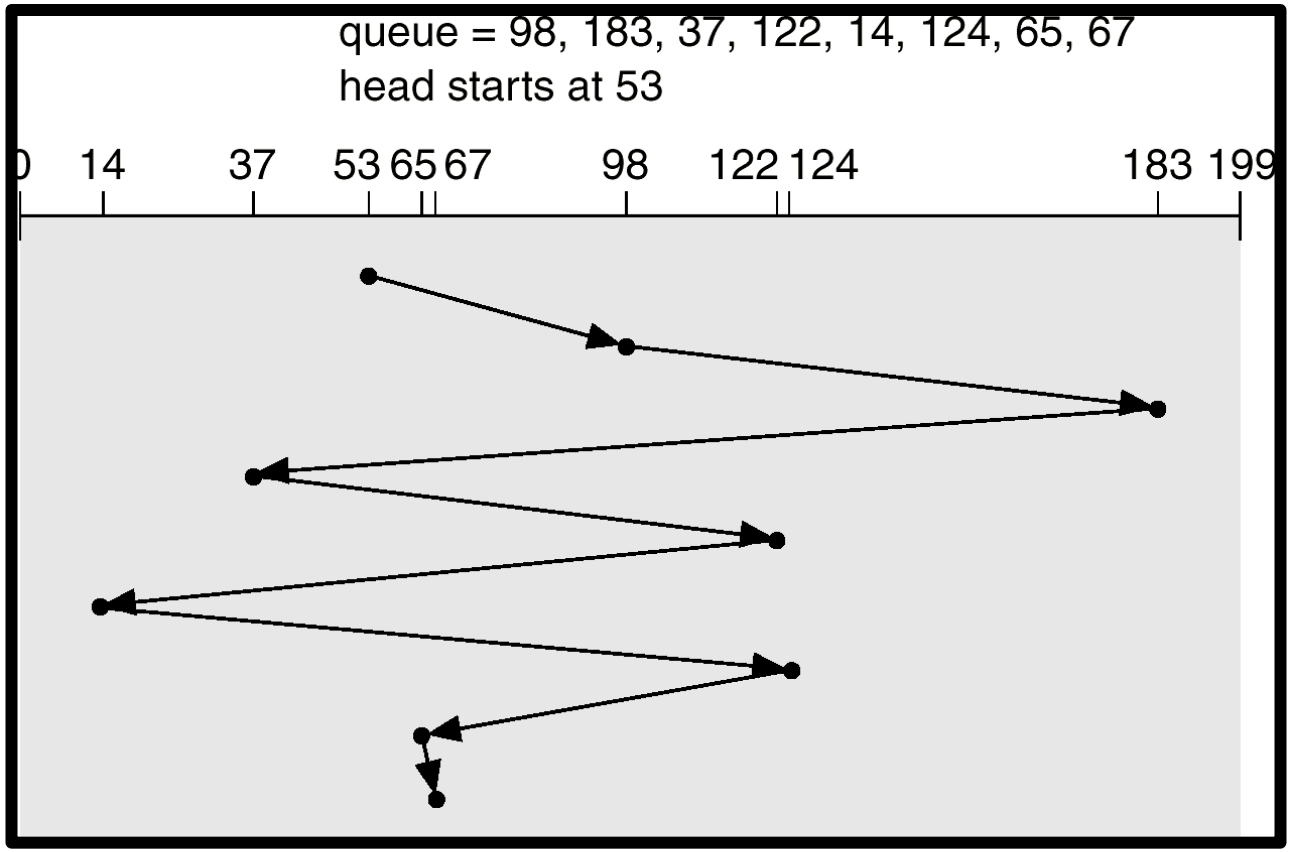
- The disk is accessed as linear array of sectors
- How to identify a sector?
  - Sectors are located by an integer index
  - Controller translates from address to physical position
  - Shields OS from structure of disk



Disk scheduling

- Note that drive controllers have small buffers and can manage a queue of I/O requests (of varying “depth”)
- Several algorithms exist to schedule the servicing of disk I/O requests
  - FCFS
  - SSTF (shortest seek time first)
  - Elevator schedulers (Scan, C-Scan)
- We illustrate them with an I/O request queue (cylinders are between 0-199):
  - **queue = 98, 183, 37, 122, 14, 124, 65, 67**
  - **head starts at 53**

FCFS

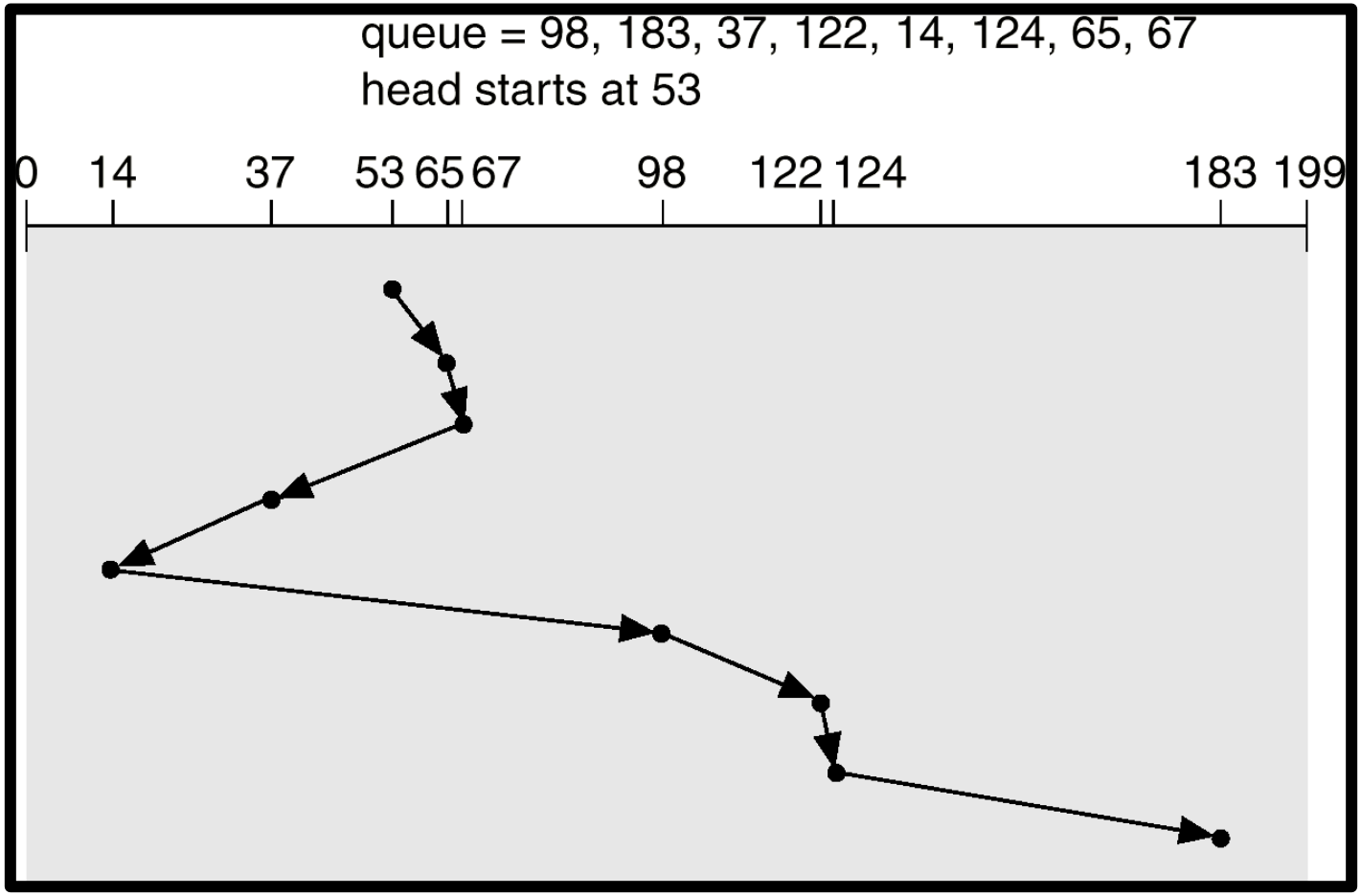


Total head movement: 640 cylinders

SSTF

- Selects the request with the minimum seek time from the current head position
- SSTF scheduling is a form of SJF scheduling (potential starvation)

SSTF



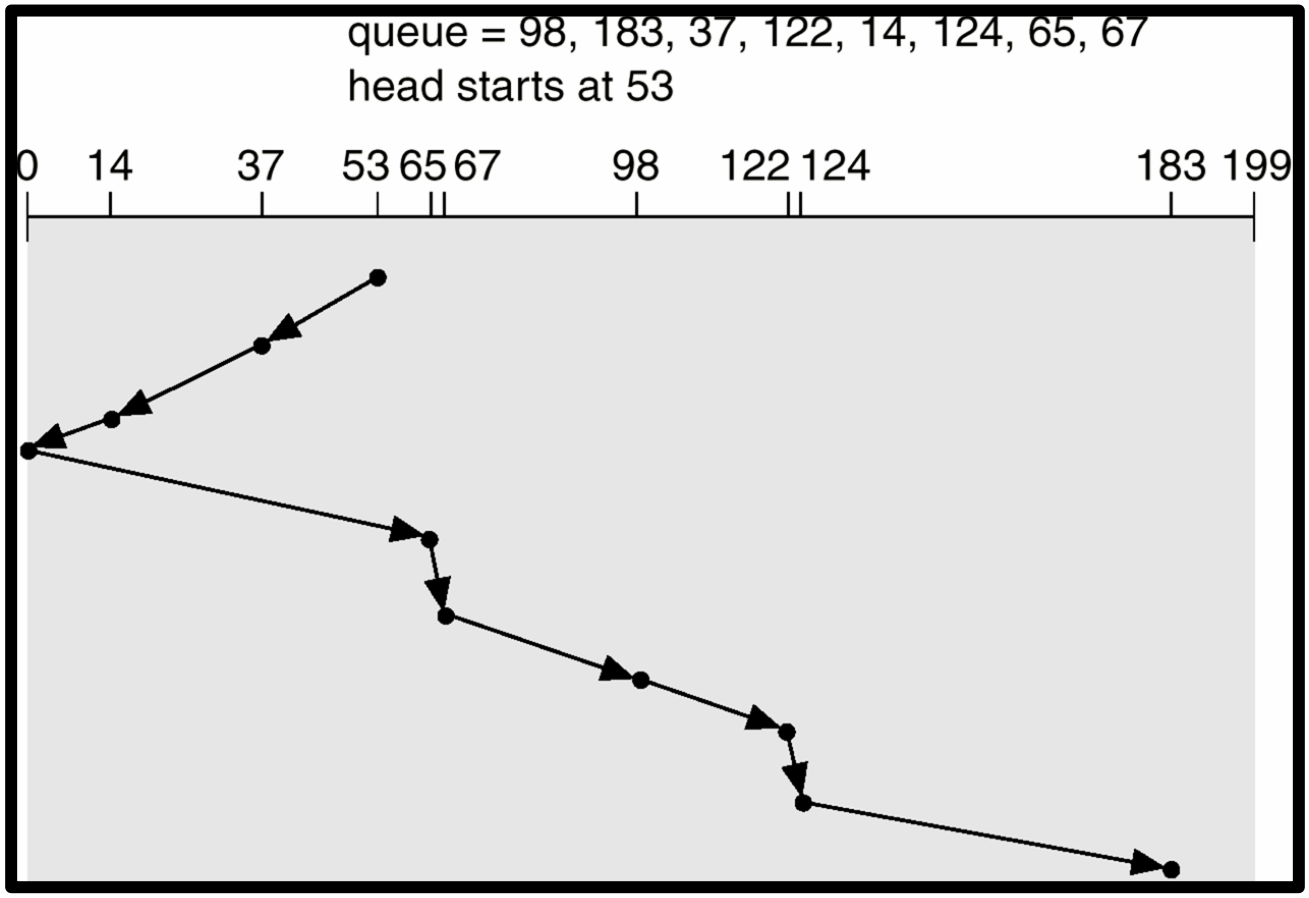
Total head movement: 236 cylinders

Elevator Algorithms

- Algorithms based on the common elevator principle
- Various combinations of Elevator algorithms:
  - Service in both directions or in only one direction
  - Go until last cylinder or until last I/O request.

SCAN

- The disk arm starts at one end of the disk, and moves toward the other end, servicing requests until reaching the other end

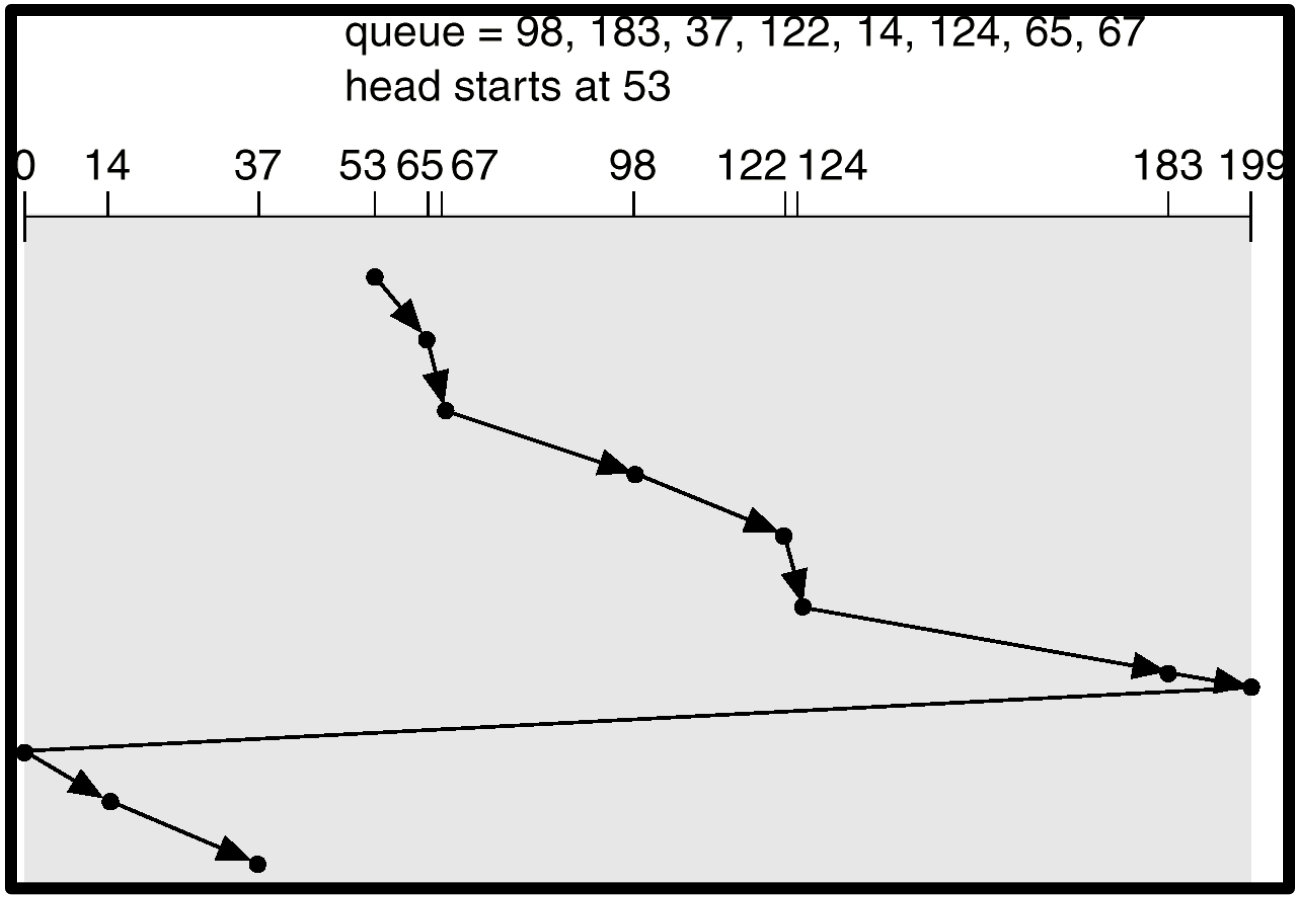


Total head movement: 208 cylinders

C-SCAN

- Provides a more uniform wait time than SCAN
- The head moves from one end of the disk to the other, servicing requests as it goes. When it reaches the other end, however, it immediately returns to the beginning of the disk, without servicing any requests on the return trip
- Treats the cylinders as a circular list that wraps around from the last cylinder to the first one
- C-SCAN may reduce the average waiting time for requests compared to SCAN, yet worse performance in scenarios where there are many requests queued at the end of the disk queue

C-SCAN


queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53
0
14
37
53
65
67
98
122
124
183
199

Selecting a disk scheduler

- SSTF is common and has a natural appeal
- SCAN and C-SCAN perform better for systems that place a heavy load on the disk
- Performance depends on various factors, e.g.,
  - # and types of requests
  - File allocation method

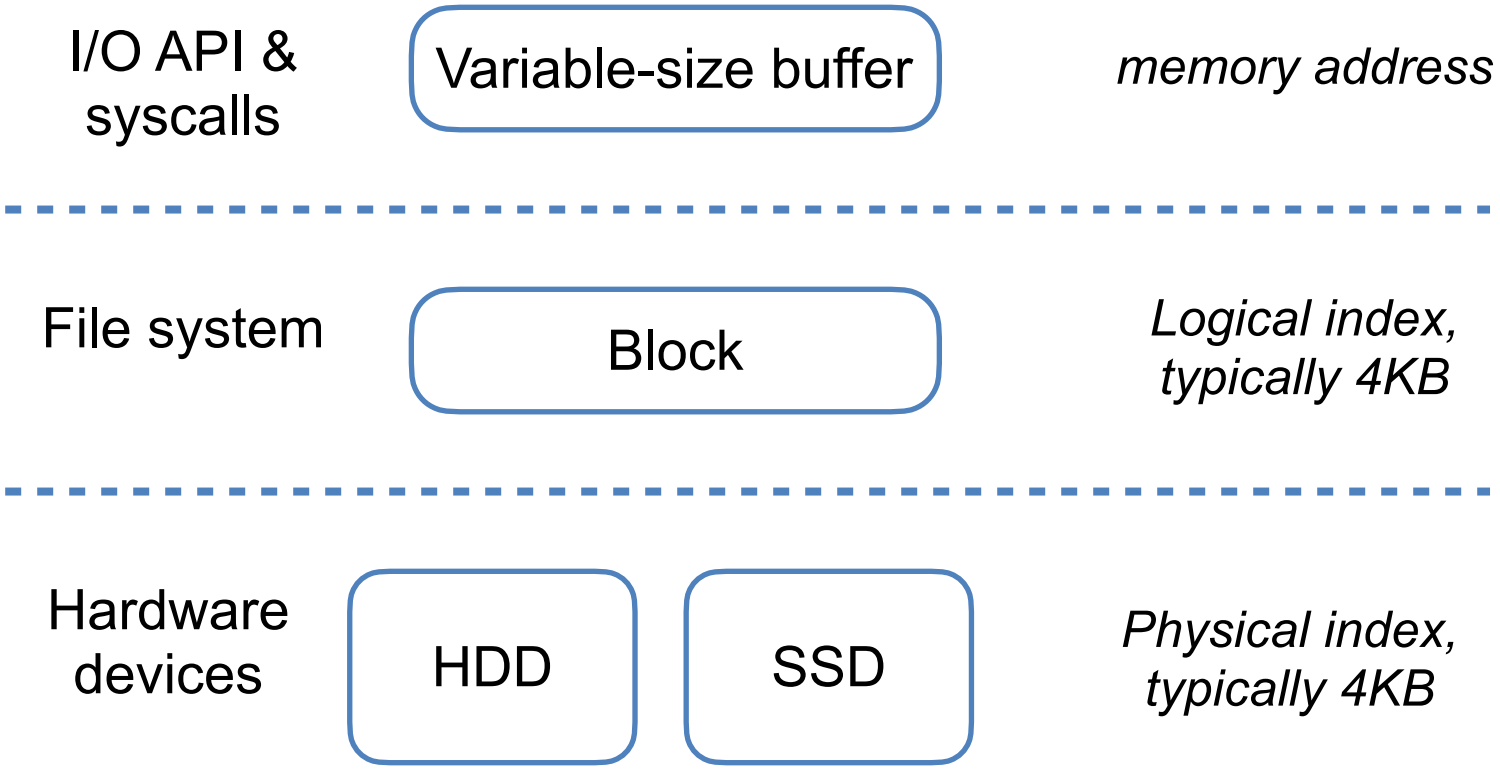
Discussion Questions

Why does the SCAN (Elevator) disk scheduling algorithm outperform FCFS under high load?

A. It uses predictive algorithms based on access frequency.
B. It assigns priorities to I/O requests based on deadlines.
C. It minimizes seek distance by scanning in one direction before reversing, reducing head movement.
D. It queues all reads before writes to simplify scheduling.

Interfacing Storage Device with File System

From storage to file systems



Building a file system

- File system: layer of OS that transforms block interface of disks (or other block devices) into files, directories, etc
- Features:
  - Naming: find file by name (not block numbers)
  - Organization: Organize file names with directories
  - Mapping: map files to blocks
  - Protection: enforce access restrictions
  - Reliability: keep files intact despite crashes, hardware failures, etc.

File system abstractions

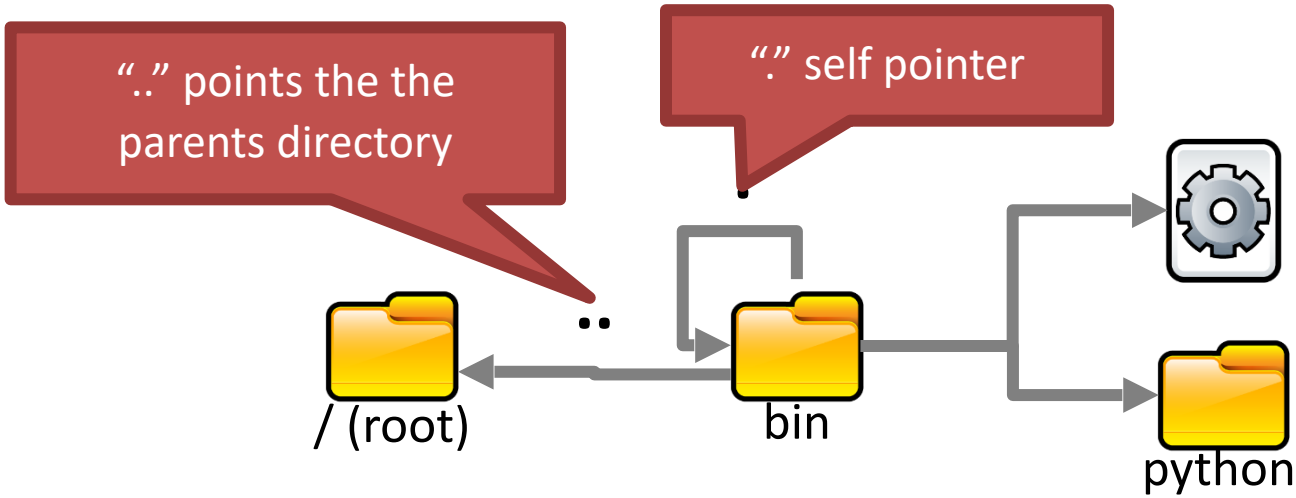
- How do users/user programs interact with the file system?
  - Files: user-visible group of blocks arranged sequentially in logical space
  - Directories: user-visible index mapping names to files
  - Links: a pointer that associate a file or directory with a specific name in a specific location in the directory tree
- File system design issues:
  - How are files and directories stored?
  - How is storage space managed?

Files

- A file is a composed of two components
- The file data itself
  - One or more blocks (sectors) of binary data
  - A file can contain anything
- Meta-data about the file
  - Name, total size
  - Created time, modified time, access time
  - Hidden or system file
  - Owner and owner's group
  - Permissions: read/write/execute

Directories

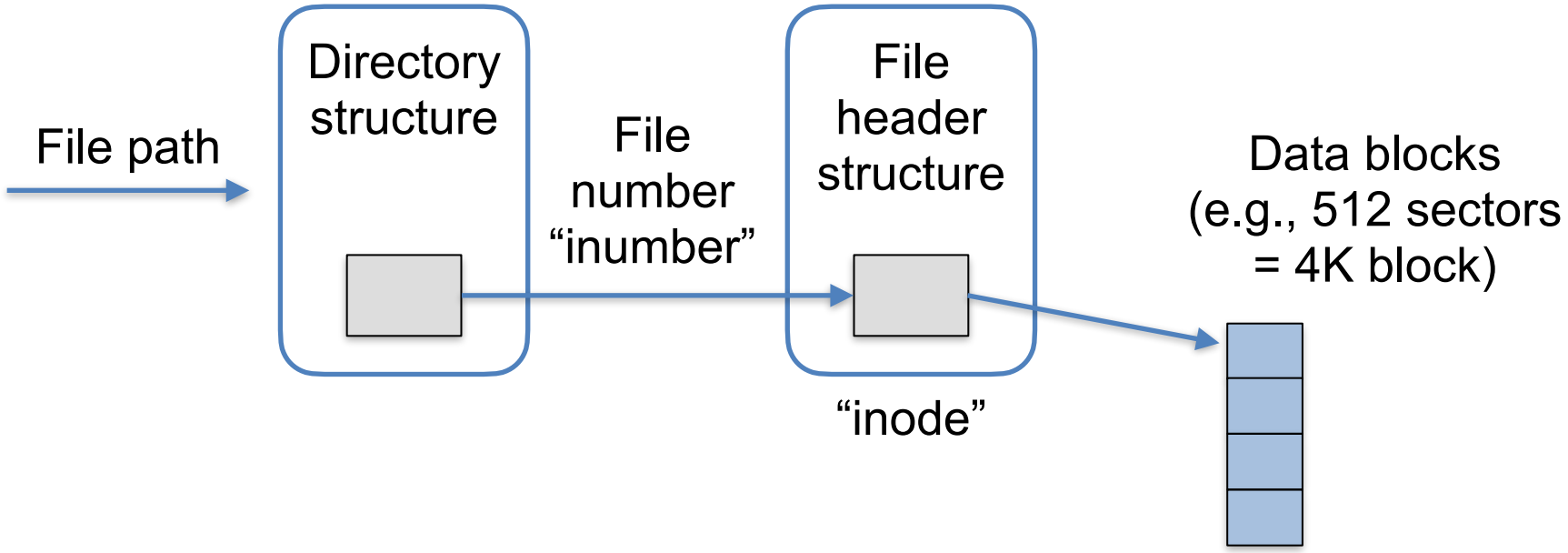
- Traditionally, file systems have used a hierarchical, tree-structured namespace
  - Directories are objects that contain other objects
    - i.e. a directory may (or may not) have children
  - Files are leaves in the tree
- By default, directories contain at least two entries



More on Directories

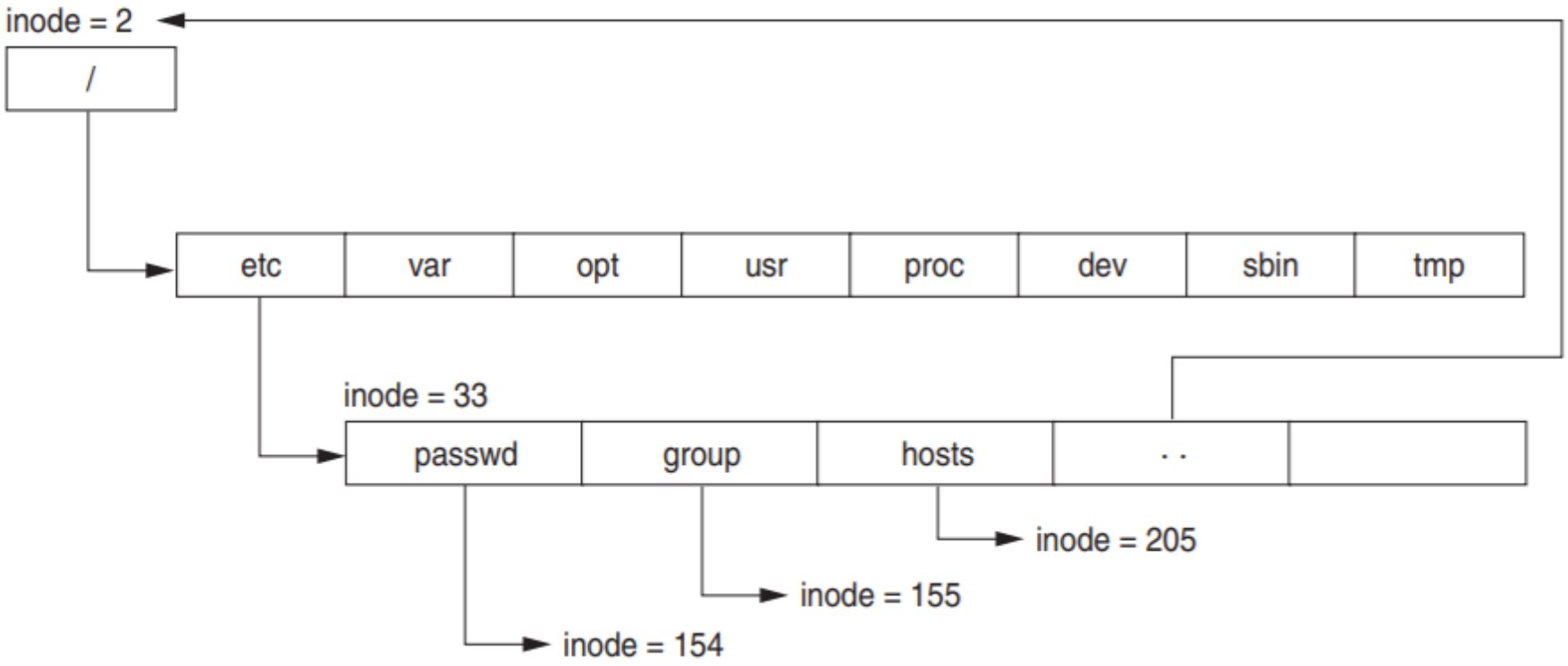
- Directories have associated meta-data
  - Name, number of entries
  - Created time, modified time, access time
  - Permissions (read/write), owner, and group
- The file system must encode directories and store them on the disk
  - Typically, directories are stored as a special type of file
  - Directory file contains a list of entries inside the directory, plus some meta-data for each entry

Components of a file system



UNIX Directory Hierarchy

- Each directory file has multiple entries



Core FS Objects

- **Superblock**: global metadata of a file system
  - Volume size, # of nodes, free list
  - Many file systems put this as first block of partition
- **inode** (index node): represents one file
  - Keeps metadata as well as pointers to data blocks
  - File types (directory or file), File size, Permission, Attributes
- **dentry** (directory entry): name-to-inode mapping
  - Directory itself is stored as a file, so it has a corresponding inode
- **File object**: data structure representing an opened file in memory
  - Keeps pointer to inode (or dentry), access permissions, and file offset

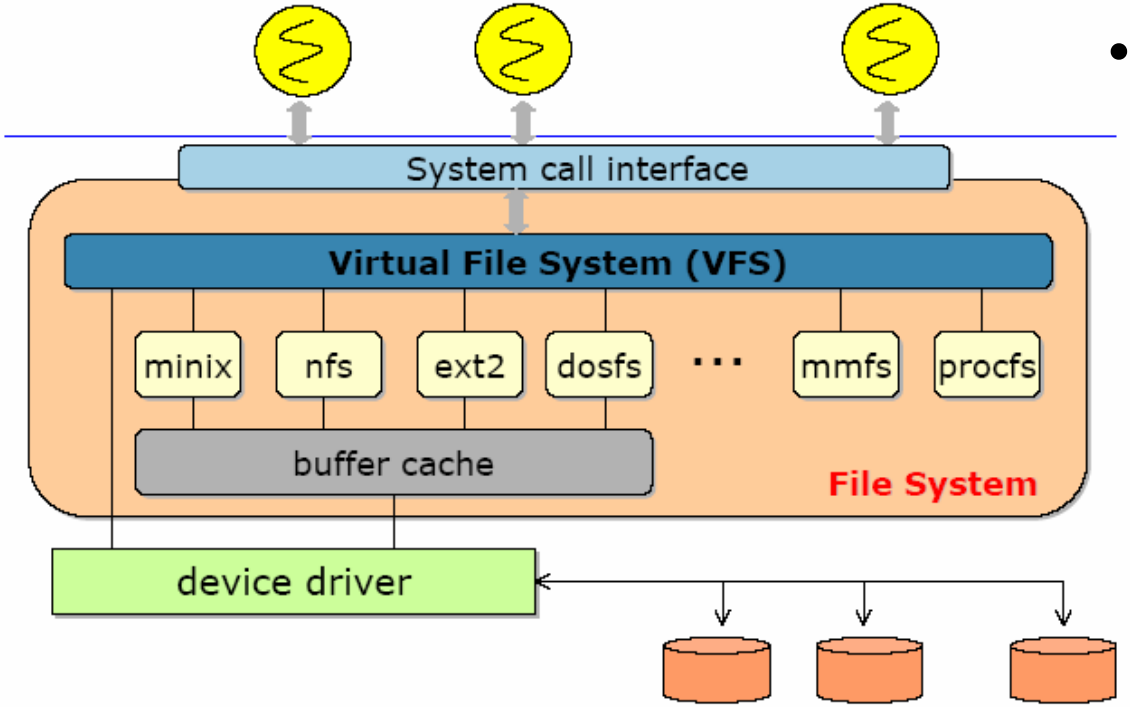
Discussion Questions

Which best describes the role of an inode in a Unix-style file system?

A. It maps file paths directly to physical sectors.
B. It holds the actual file data in linked blocks.
C. It contains metadata and pointers to data blocks but not file names.
D. It maintains user access permissions and directory paths.

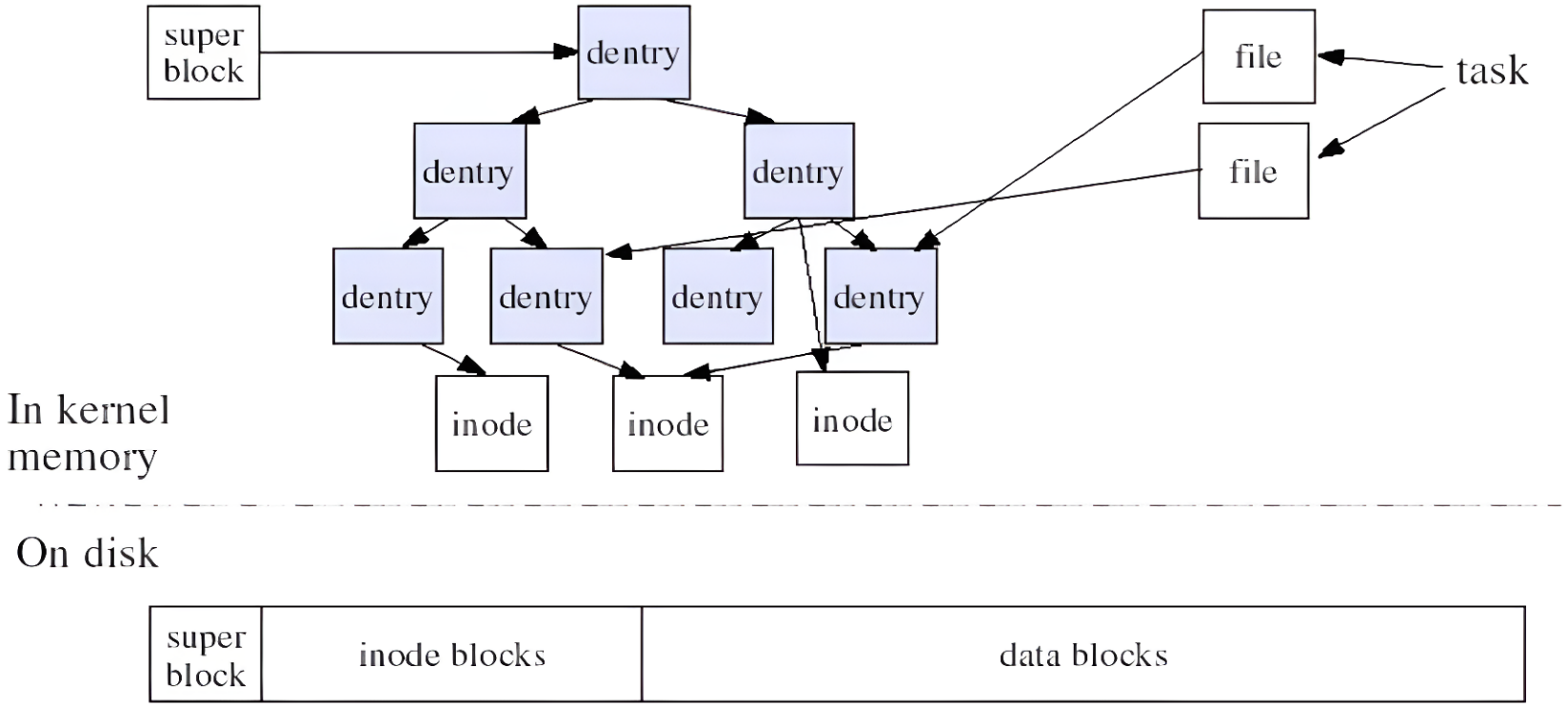
Virtual File System

- Problem: the OS may mount several partitions containing different underlying file systems
- **VFS:** Abstracts away concrete file system implementation
  - Provide the same interface to user programs (POSIX APIs)
  - Provide a kernel abstraction for all different file system implementations



- Linux vfs interface includes:
  - creat(name)
  - open(name, how)
  - read(fd, buf, len)
  - write(fd, buf, len)
  - sync(fd)
  - seek(fd, pos)
  - close(fd)
  - unlink(name)

Virtual File System: Linux



- Superblocks, inodes and dentries are stored on-disk
  - and cached in main memory when accessed
- File object is only in memory

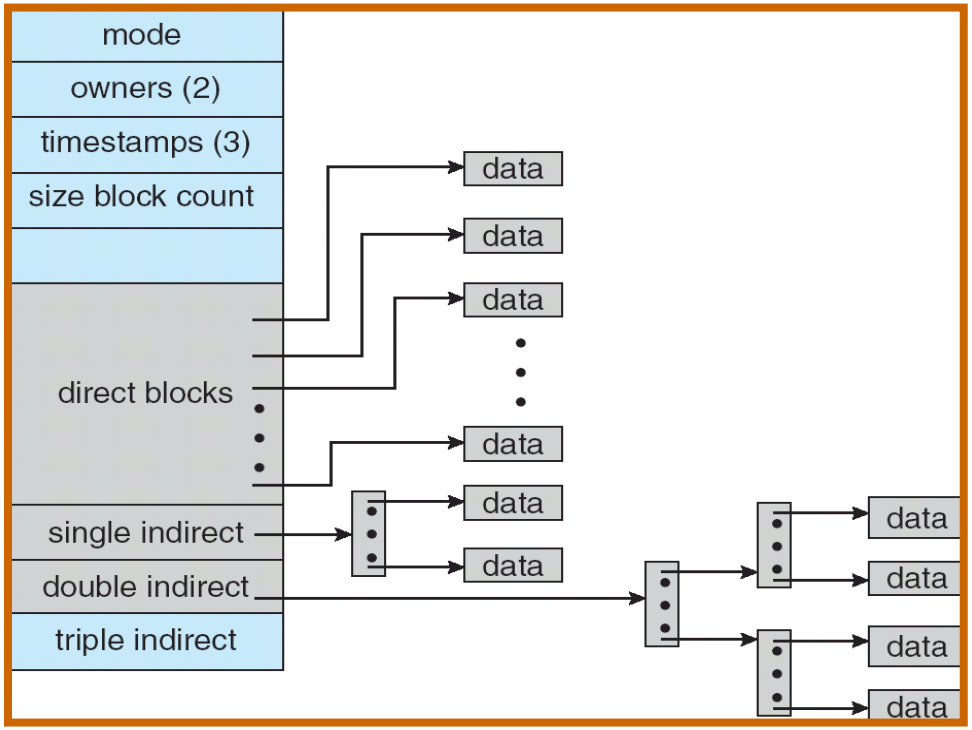
Discussion Questions

How does the Virtual File System (VFS) enable portability across different file systems?

A. By dynamically linking each file system at boot time.
B. By storing all files in a virtual block pool shared by all devices.
C. By exposing a common POSIX API and kernel abstraction, regardless of underlying implementations.
D. By converting all file access to memory-mapped I/O operations.

Zooming in on inode

- inode: structure for per-file metadata (unique per file)
  - File types: directory, file
  - File size
  - Permission
  - Attributes
  - About 10-15 data-block pointers



- Indirect blocks:
  - inode only holds a small number of data block pointers (direct pointers)
  - For larger files, inode points to an indirect block containing 1024 4-byte entries in a 4K block
  - Each indirect block entry points to a data block
  - Can have multiple levels of indirect blocks for even larger files
    - Each block has 1024 ptrs: 4MB@level 2; 4GB@level 3; 4TB@level 4

Unix Inodes and Path Search

- Unix inodes do not have file path information
- Inodes describe where on disk the blocks for a file are placed

- Directory entries (dentries) map file paths to inodes
  - To open "/home/foo/bar.c", use superblock to find inode for "/" on disk
  - Open "/" inode, look for entry of "home"
    - This entry gives the disk block number for the inode for "home"
  - Open "home" inode, look for entry of "foo"
  - Open "foo" inode, look for entry of "bar.c"
  - Open "bar.c" inode
    - The inode says where first data block is on disk
  - Read that data block into memory to access the data in the file

- This is why we have *open()* in addition to *read()* and *write()*

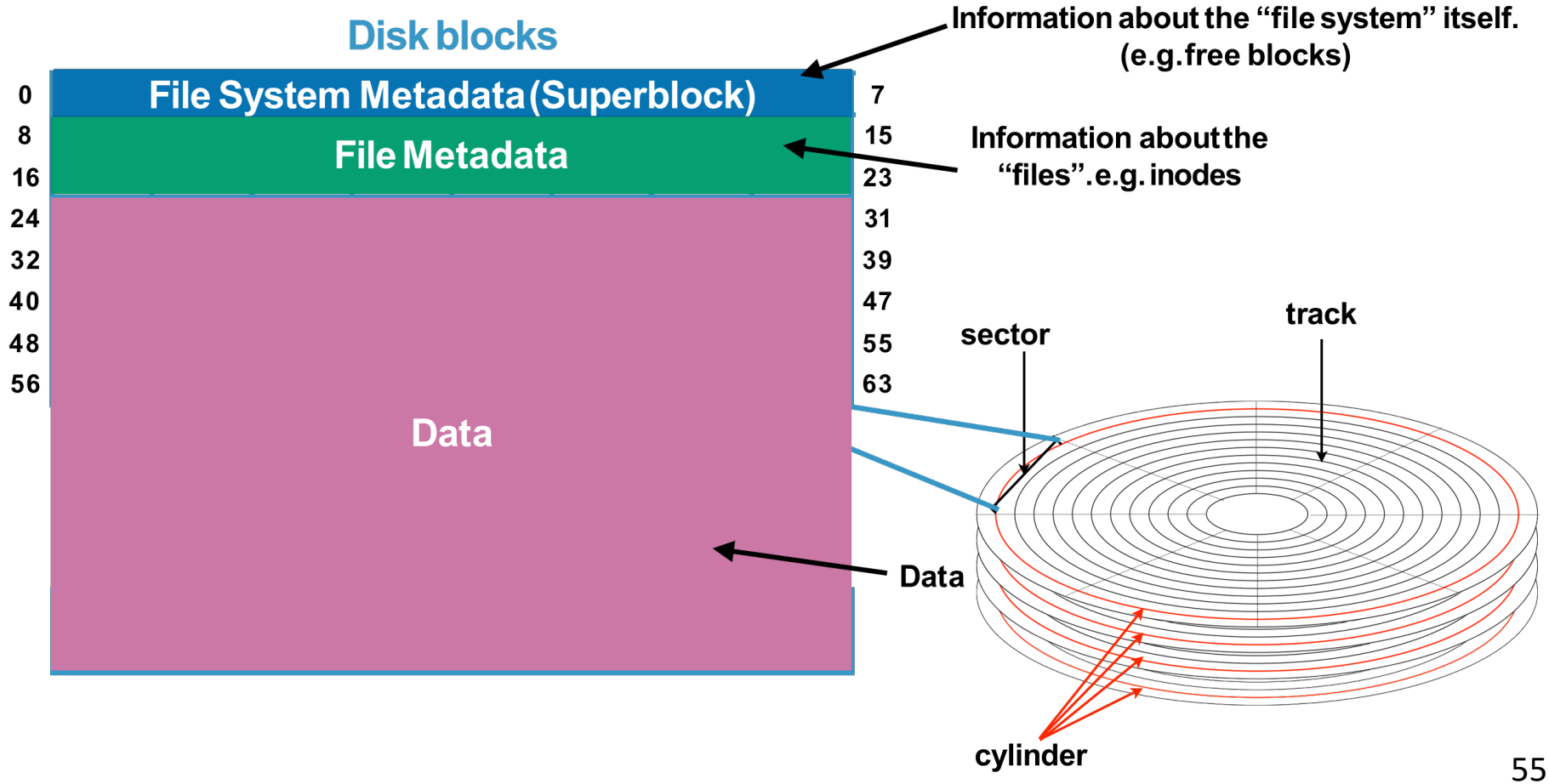
Discussion Questions

Why is open() necessary in Unix before calling read() on a file?

A. read() requires a file's logical block address as input.
B. File systems need to cache the entire file before read.
C. open() resolves the pathname to an inode and sets up a file object with metadata for access.
D. read() only works on directories unless opened first.

Old UNIX File System (UFS)

- Ext (extended file system) in Linux was inspired by UFS
- UFS mapping to LBA



Problems of UFS

- Poor locality
  - inodes are far from their corresponding data
    - All inodes at the beginning of the disk, all data after that
  - inodes of directory not close together
  - Consecutive blocks of files not close together
  - Results in long seeks across the disk

- Prone to fragmentation
  - UFS chooses the first available blocks for new data
  - No attempt is made to keep the blocks of a file contiguous